

HR DATA WAREHOUSE & ANALYTICS PROJECT

Technical & Business Documentation

HR & Workforce Analytics Domain

Project Name: HR Data Warehouse and Analytics

Prepared by: Group 3

Technologies: SQL Server, SSMS, T-SQL, SQL Server Agent, Power BI

Date: August 25, 2026 (revision)

Project Type: Data Warehouse & Power BI Report

Repository: github.com/rahmahamdy590-del/HR-DataWarehouse-Project

Team Members

Team Lead: Rahma Hamdy

Angham Atef — Database creation & Bronze load layer

Samah Ali — Bronze data-quality profiling

Hend Mostafa — Silver Layer

Rahma Hamdy — Gold Layer

Raghad Atef — Diagrams

This documentation describes the project as actually implemented; every point where the real implementation diverges from the originally planned design or diagrams is called out explicitly rather than hidden.

Table of Contents

Table of Contents	2
Executive Summary	5
Project Overview	5
Business Problem	5
Business Value	5
Target Users.....	5
Business Benefits.....	5
Expected Outcomes.....	6
Project Objectives.....	6
Business Requirements	6
Technology Stack.....	7
Repository / Folder Structure	7
Dataset Description	7
Key Fields and Business Meaning.....	8
Data Quality (discovered during Bronze-layer profiling).....	8
Installation Guide	8
SQL Script Inventory	9
Architecture Overview.....	9
Medallion Architecture.....	9
Layer Responsibilities	10
Bronze Layer	11
Bronze Layer Data-Quality Profiling	11
Business Notes.....	11
Silver Layer	12
Tables Produced	12
Cleaning and Standardization.....	12
Normalization and Business Rules.....	12
Deduplication	12
Dimension and Fact Staging	12
Expected Output.....	12
Gold Layer.....	13
Key Implementation Change: Gold Tables → Gold Views.....	13
Why Views Instead of Physical Gold Tables	13
Gold Objects (Current Implementation)	13
Important Structural Decisions.....	13

Fact View Population.....	13
Built-In Validation.....	13
Expected Output.....	14
ETL Pipeline.....	14
ETL Automation (SQL Server Agent).....	14
Job Steps (in execution order).....	14
Data Lineage.....	15
Why Each Transformation Exists.....	16
Silver-to-Gold Mapping.....	16
Star Schema.....	16
Data Dictionary — Current Gold Views.....	17
gold.v_dim_employee.....	17
gold.v_dim_position.....	17
gold.v_dim_manager.....	18
gold.v_dim_performance.....	18
gold.v_dim_recruitment.....	18
gold.v_fact_hr_snapshot.....	18
SQL Design Decisions.....	19
Docker / Containerization.....	Error! Bookmark not defined.
Power BI Semantic Model.....	19
Relationships.....	20
DAX Measures Captured.....	20
KPIs, Design, and Business Narrative.....	21
Dashboard Tour.....	21
Page 1 — Workforce.....	21
Page 2 — Turnover & Retention Rate.....	22
Page 3 — Recruitment & Hiring.....	22
Page 4 — Organization Insights.....	23
Business Insights.....	24
Known Deviations from Original Design.....	24
Data Validation.....	25
1. Silver-to-Gold Row-Count Validation.....	25
2. Missing Employee Relationship Validation.....	25
3. Missing Position Relationship Validation.....	25
4. Employee Status Consistency Validation.....	26
5. Termination-Date Consistency Validation.....	26
Challenges.....	26

Lessons Learned	26
Future Improvements.....	27
Conclusion	27
Technologies Used.....	27
References	27
Contact	28
Appendix A — Complete SQL Scripts (Current Implementation).....	28
00_initialize_database.sql — Current Version	28
01a_bronze_load.sql	28
01b_bronze_profiling.sql.....	30
02_silver_transformation.sql — Current Version	33
03_gold.sql — Current Gold Views Version	36
04_sql_agent_job.sql	38
Appendix B — Full DAX Query Definitions	52
DAX Query File: Query 1.dax	52
DAX Query File: Query 2.dax	53
DAX Query File: Query 3.dax	54

Executive Summary

This project transforms a single flat HR dataset (HRDataset_v14.csv, 311 employees, 36 columns) into a three-layer data warehouse on SQL Server, following the Bronze → Silver → Gold (Medallion) architecture. The current Gold layer is implemented as a set of analytical SQL Views built on top of the Silver tables, and a four-page Power BI report consumes the Gold layer for workforce, turnover, recruitment, and organizational analytics.

Key implementation update: the current version of the project uses Gold Views rather than physical Gold tables. Any earlier description of dim_*/fact_* objects as physical Gold tables, or of Surrogate Keys inside Gold, is historical. The current SQL implementation in Appendix A is the authoritative reference, and it is reflected consistently throughout this document.

The warehouse is built through a repeatable, five-step pipeline (00 → 01a → 01b → 02 → 03) that takes the raw CSV from zero to a clean Silver layer and a business-ready Gold analytical layer, automated end-to-end through SQL Server Agent. A four-page Power BI dashboard (Workforce, Turnover & Retention, Recruitment & Hiring, and Organization Insights) delivers self-service reporting against this model.

Project Overview

Business Problem

HR departments accumulate large volumes of employee data — demographics, compensation, performance, recruitment source, tenure, and termination reasons — but this data is typically kept in a single flat operational file that is not prepared for analysis. Without a proper analytical model, HR stakeholders cannot confidently answer questions such as: Which departments are losing the most employees? Is compensation equitable across departments? Which recruitment channels produce the best long-term hires?

Business Value

The project converts the single flat HR dataset (HRDataset_v14.csv, 311 employees, 36 columns) into a governed, three-layer data warehouse on SQL Server, with a Power BI semantic model on top, enabling reliable, repeatable, self-service HR reporting instead of one-off manual analysis on separate spreadsheets.

Target Users

- HR business partners and managers (workforce, turnover, and retention reporting).
- A single source of truth (the Silver layer) for all HR reporting, with a Gold Views layer that presents the data in analysis-ready form without duplicating it into separate physical Gold tables.
- An analytical Gold layer built as Views on top of Silver, which preserves presentation logic, minimizes storage duplication, and lets Power BI consume analysis-ready data directly.
- Data and BI teams responsible for maintaining the warehouse and dashboard going forward.

Business Benefits

- A single source of truth (the Silver layer) for all HR reporting, replacing ad hoc spreadsheet manipulation.
- A practical, rebuildable pipeline (00 → 01a → 01b → 02 → 03) that takes the raw CSV from zero to a cleaned Silver layer, then delivers a Gold Views layer representing the logical star schema, plus a Power BI report.
- Scheduled, automatic rebuilds via SQL Server Agent instead of running scripts manually.

- A four-page interactive Power BI dashboard that answers the specific business questions defined in the requirements document.

Expected Outcomes

A practical, rebuildable pipeline (00 → 01a → 01b → 02 → 03) that takes the raw CSV from zero to a fully modeled star schema, plus a Power BI report presenting workforce composition, turnover and retention, recruitment effectiveness, and organizational insights.

Project Objectives

- Build a complete three-layer data warehouse (Bronze/Silver/Gold) on SQL Server, starting from a single raw HR CSV file.
- Apply Medallion architecture methodology: Bronze = raw and unmodified, Silver = cleaned and conformed, Gold = presentation-ready.
- Comprehensively profile raw data quality before writing any cleaning logic, and let the profiling results drive every transformation decision.
- Design a star schema (facts and dimensions) that directly supports the business questions defined for the HR domain.
- Fully automate the pipeline using SQL Server Agent so that it can be re-run without manual intervention.
- Build a Power BI report connected to the Gold layer in Import mode, with real DAX measures answering specific business questions.
- Document the project to a professional standard suitable for a portfolio deliverable.

Business Requirements

The dashboard and data model were built to answer the following HR analytics requirement areas:

Requirement Area	Business Question	Answered In
Employee Analysis	How many employees are active/terminated? What is the gender distribution?	Workforce page
Department Analysis	Which departments lead in headcount, hiring, and turnover?	Workforce, Turnover, Recruitment pages
Salary Analysis	How does salary vary by department? Min/max/average?	Workforce, Turnover pages
Performance Analysis	Which managers supervise the most satisfied employees?	Organization Insights page
Recruitment Analysis	Which recruitment sources produce the most hires?	Recruitment & Hiring page
Attrition Analysis	Overall turnover rate and top termination reasons?	Turnover & Retention page
Retention Analysis	Overall retention rate and its trend?	Turnover & Retention page
Manager Performance	Which managers supervise the largest / most satisfied teams?	Organization Insights page
Employee Satisfaction	Average satisfaction and engagement scores?	Workforce, Organization Insights pages
Workforce Demographics	Distribution of marital status / gender / recruitment source?	Workforce page

Technology Stack

Every technology in this stack was a deliberate choice, not a default; the rationale for each is given below.

Technology	Role	Why It Was Chosen
Microsoft SQL Server	Hosts the HR_Project database and the bronze/silver/gold schemas	Industry-standard relational engine, with native schema support, BULK INSERT, and SQL Server Agent scheduling — everything needed for a local, single-engine data warehouse
SSMS	Development and execution environment	Native T-SQL authoring and debugging, plus SQL Server Agent job management
T-SQL	All ETL logic	Keeps the heavy cleaning and modeling logic close to the data — no need for a separate ETL engine at this scale
SQL Server Agent	Schedules the 5-step ETL pipeline	Native scheduling; enforces immediate-stop behavior on failure between interdependent steps
Power BI Desktop	Semantic model + DAX + 4-page report	Enterprise-grade DAX for time-intelligence measures (turnover/retention) and native star-schema modeling
CSV / Excel	Raw source format	Matches the realistic scenario of HR data exported from an HRIS system as a flat file
GitHub	Version control and delivery	Single source of truth for scripts, documentation, and the Power BI file

Repository / Folder Structure

The repository follows the structure required by the project requirements document:

```
project root/ ├── README.md ├── data/ | ├── HRDataset_v14.csv | └── README.md ├── scripts/ | ├──
00_initialize_database.sql | ├── 01a_bronze_load.sql | ├── 01b_bronze_profiling.sql | ├── 02_silver_transformation.sql | ├──
03_gold.sql | └── 04_sql_agent_job.sql ├── docs/ | ├── architecture.jpeg | ├── data_flow.jpeg | ├── star_schema.jpeg | └──
technical report (this document) └── power bi/ ├── HR_Analysis_Dashboard_project.pbix ├── pbi_model_view.jpeg └──
dashboard_1.png ... dashboard_4.png
```

Folder	Purpose
data/	The raw CSV file, exactly as downloaded, with a note on its source
scripts/	All T-SQL scripts, numbered in required execution order
docs/	Architecture, lineage, and star-schema diagrams, plus this technical report
power bi/	The .pbix file and screenshots of each report page and the model view

Dataset Description

Property	Value
Source	Human Resources Data Set (HRDataset_v14) — Kaggle
Format	Single flat CSV file
Row Count	311 employee records
Column Count	36
Grain	One row per employee

Key Fields and Business Meaning

Field	Business Meaning
EmpID	Unique employee identifier (business key)
Employee_Name	Full employee name, stored as "Last, First" in the raw file
DeptID / Department	The department the employee belongs to
PositionID / Position	Job title / role
ManagerID / ManagerName	The employee's direct manager
Salary	Annual salary
DateofHire / DateofTermination	Employment start and end dates
EmploymentStatus / Termd / TermReason	Whether the employee is active, and if not, why
PerformanceScore	Latest performance rating
EngagementSurvey / EmpSatisfaction	Engagement score (0–5) and satisfaction score (1–5)
RecruitmentSource	The channel through which the employee was hired
DaysLateLast30 / Absences	Attendance and discipline metrics
Sex, MaritalDesc, CitizenDesc, HispanicLatino, RaceDesc, State, DOB	Workforce demographic attributes

Data Quality (discovered during Bronze-layer profiling)

The raw file is generally well-formed as an educational dataset, but the profiling process specifically checks for: duplicate employee IDs, duplicate employee names, missing name/department/position/manager ID, invalid salaries (≤ 0), out-of-range engagement scores (outside 0–5) and satisfaction scores (outside 1–5), negative absences/lateness, orphaned manager IDs (managers that do not appear as an employee row themselves, including cases where the ID is stored as "22.0" instead of 22), invalid hire/termination dates, and missing performance scores.

The warehouse follows the classic layered pattern Bronze → Silver → Gold, entirely within a single SQL Server instance. Bronze stores the source as-is, Silver holds the cleaned tables and primary keys, and Gold exposes analytical Views on top of Silver.

Installation Guide

- Install SQL Server (Developer or Express edition is sufficient for this data volume).
- Install SQL Server Management Studio (SSMS) to author and run the scripts.
- Clone or download this repository to your local machine.
- Place the raw CSV file at the path referenced inside 01a_bronze_load.sql, or update the BULK INSERT ... FROM path to point to your local copy of data/HRDataset_v14.csv.
- Run the scripts in SSMS in the exact order: 00 → 01a → 01b → 02 → 03.
- (Optional) Deploy 04_sql_agent_job.sql to an instance running SQL Server Agent to schedule the full pipeline.
- Open Power BI Desktop and open HR_Analysis_Dashboard_project.pbix.
- In Power BI, redirect the data-source connections to your HR_Project gold schema (Transform Data → Data Source Settings) and refresh.

Known limitation: the delivered script contains a hard-coded local Windows path —

C:\Users\pc\Documents\GitHub\... — for the BULK INSERT source file. This path must be changed to your own machine's path before running. It has not been parameterized in the current implementation; see Future Improvements.

SQL Script Inventory

This chapter summarizes the purpose, dependencies, and outputs of each script. The full script content is reproduced in Appendix A — this section explains why and how each script works rather than restating it line by line.

Script	Purpose	Depends On	Objects Created
00_initialize_database.sql	Create the database and the three schema namespaces	—	HR_Project database; bronze/silver/gold schemas
01a_bronze_load.sql	Load the raw CSV unmodified via BULK INSERT	00	bronze.hr_data
01b_bronze_profiling.sql	Run 15 diagnostic data-quality checks	01a	No persisted objects (query results only)
02_silver_transformation.sql	Clean, type-cast, standardize, and conform	01a	8 tables in the silver schema
03_gold.sql	Build the analytical Gold Views and run validation queries	02	5 dimension Views + 1 fact View
04_sql_agent_job.sql	Automate and schedule everything above	00–03	One SQL Server Agent job, 5 steps, daily schedule

Architecture Overview

Medallion Architecture

The high-level architecture follows the standard Bronze → Silver → Gold layering, entirely inside a single SQL Server instance. Bronze stores the source as-is; Silver holds the cleaned tables and primary keys; Gold exposes business-ready analytical Views on top of Silver.

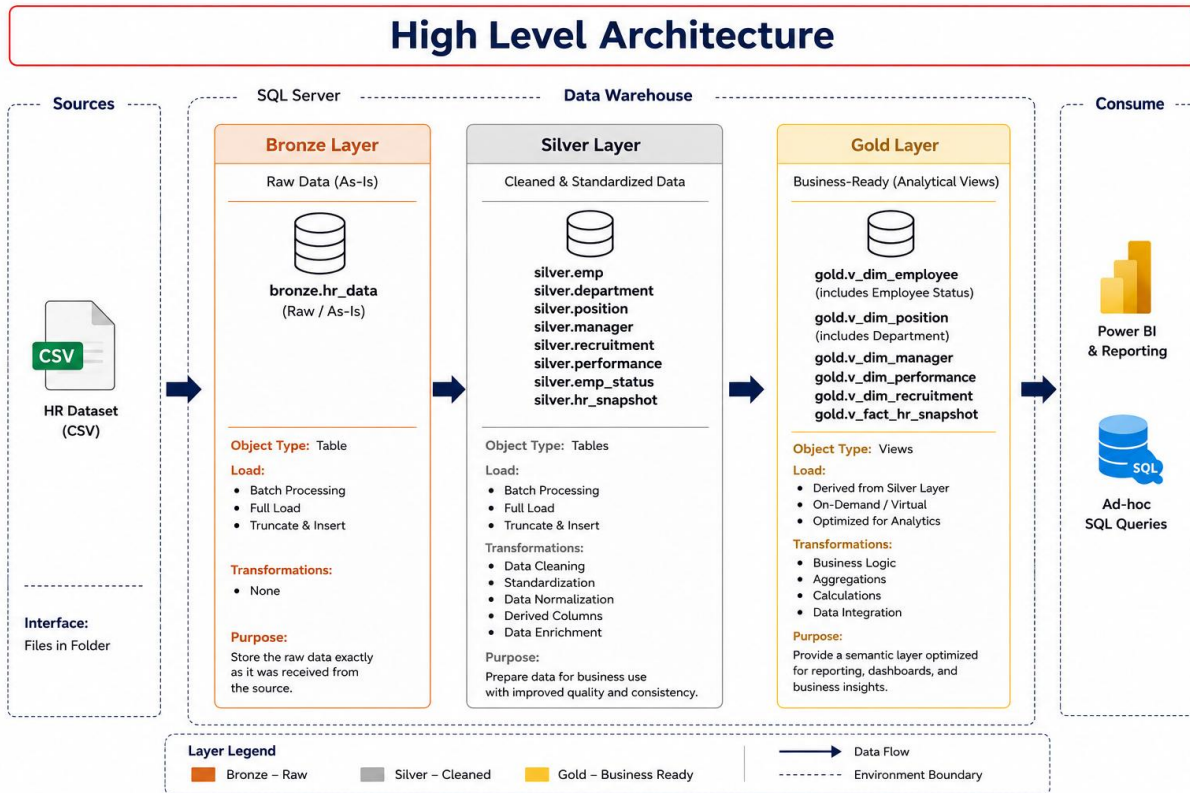


Figure 1 — High-level architecture: Source → Bronze → Silver → Gold Views (Business-Ready Analytical Views) → Power BI / Ad-hoc SQL.

Layer Responsibilities

Layer	Responsibility
Bronze	Raw, unmodified copy of the source CSV, loaded via BULK INSERT. Every column is VARCHAR(MAX) — no type casting, no cleaning.
Silver	Cleaning, type conversion, standardization, deduplication, and conformance across 8 normalized physical tables. This is the single source of truth.
Gold	Business-ready analytical layer: 5 dimension Views + 1 fact View, built directly on top of Silver. No physical tables and no surrogate keys are created in Gold.
Power BI	Connects to the Gold layer only, in Import mode, and applies light Power Query shaping plus DAX measures for reporting.

Note: the architecture diagram above labels each layer generically as a processing stage; the current implementation executes each layer as an ordinary sequential T-SQL script/batch, not as a CREATE PROCEDURE object. This distinction is repeated in "Known Deviations from Original Design."

Bronze Layer

Purpose: ingest the raw HR CSV file into SQL Server exactly as delivered, with no transformation, so an original, re-runnable copy of the source is always available.

- Raw storage: `bronze.hr_data` — all 36 columns are typed as `VARCHAR(MAX)`, deliberately avoiding any type coercion at this stage.
- Bulk load: `BULK INSERT` with `FORMAT='CSV'`, `FIRSTROW=2` (skips the header row), `FIELDQUOTE='\"'`, `FIELDTERMINATOR=','`, `ROWTERMINATOR='0x0d0a'`, `CODEPAGE='65001'` (UTF-8), and `TABLOCK` for load performance.
- CSV import: the source file is `HRDataset_v14.csv`, referenced via a local Windows file path.
- Profiling: handled separately in `01b_bronze_profiling.sql` — the Bronze layer itself performs no validation.
- Expected output: 311 rows, 36 columns, verified immediately after load via two `COUNT(*)` sanity checks.

Bronze Layer Data-Quality Profiling

`01b_bronze_profiling.sql` runs 15 independent diagnostic checks against `bronze.hr_data`:

#	Check	What It Detects
1	Duplicate Employee IDs	The same EmpID appearing more than once
2	Missing Employee Name	Employee_Name blank/NULL
3	Missing Department	Department blank/NULL
4	Missing Position	Position blank/NULL
5	Missing Manager ID	ManagerID blank/NULL
6	Invalid Salary	Salary <= 0
7	Invalid Engagement Score	Score outside the 0–5 range
8	Invalid Employee Satisfaction	Score outside the 1–5 range
9	Negative Absences	Absences < 0
10	Negative Days Late	DaysLateLast30 < 0
11	Duplicate Employee Names	The same Employee_Name appearing more than once
12	Orphaned Manager IDs	ManagerID with no matching EmpID (including values like '22.0')
13	Invalid Hire Date	DateofHire does not convert to a valid date
14	Invalid Termination Date	DateofTermination does not convert (when not blank)
15	Missing Performance Score	PerformanceScore blank/NULL

Business Notes

The orphaned Manager ID check (#12) is the single most consequential finding in this project — it directly drives a design fix carried into the Silver layer (casting `ManagerID` through `FLOAT` before `INT`), followed by hand-written verification queries at the end of `03_gold.sql` that trace a specific manager ("Webster Butler") from Silver back to Bronze to confirm the root cause.

No fixed counts are hard-coded in this documentation, because they depend on the actual data in the connected environment; the profiling queries themselves are the deliverable and must be executed to obtain current figures.

Silver Layer

Purpose: convert the untyped Bronze copy into a clean, standardized, deduplicated layer built on business keys — the single source of truth for everything downstream.

Tables Produced

silver.emp, silver.position, silver.manager, silver.department, silver.recruitment, silver.performance, silver.emp_status, and the wide silver.hr_snapshot table (one row per employee, carrying every business foreign key plus every measure).

Cleaning and Standardization

- All text fields: UPPER(TRIM(...)) to ensure consistent casing with no leading/trailing whitespace.
- Employee_Name: reformatted from "Last, First" to "First Last".
- All numeric/date fields: TRY_CAST / TRY_CONVERT, which turns unparseable values into NULL instead of failing the entire load.

Normalization and Business Rules

- RecruitmentSource: blank/NULL values are standardized to "UNKNOWN" rather than excluded.
- ManagerID: cast from FLOAT to INT so that stored values such as "22.0" parse correctly.
- FromDiversityJobFairID and every numeric measure in hr_snapshot: COALESCE to 0 rather than left NULL.

Deduplication

Every reference table (position, manager, department, recruitment, performance, employment status) is built via GROUP BY <business_id> + MAX(...), collapsing several raw rows that share the same ID into a single row per ID.

Dimension and Fact Staging

silver.hr_snapshot is the direct ancestor of the Gold fact View: it retains every business key (emp_id, position_id, manager_id, dept_id, performance_id, empstatus_id, RecruitmentSource) alongside every measure, ready to be joined by the Gold layer.

Expected Output

8 populated tables; a matching row count between silver.hr_snapshot and bronze.hr_data (every Bronze row produces exactly one hr_snapshot row); no explicit validation query in this script — validation is deferred to 03_gold.sql.

Note: treating "not available" as zero (via COALESCE) rather than NULL is a deliberate modeling decision with a real consequence — it means the report cannot distinguish between "zero actual absences" and "absence not recorded" without going back to the Bronze layer.

Gold Layer

Purpose: present a business-ready analytical layer on top of the cleaned Silver tables, using Views that reflect a logical star schema and serve Power BI and direct analytical querying — without creating additional physical copies of the data.

Key Implementation Change: Gold Tables → Gold Views

The originally planned design built physical Gold tables with IDENTITY surrogate keys (sk_emp, sk_position, sk_manager, sk_department, sk_recruitment, sk_performance, sk_emp_status, sk_hr_snapshot). The current implementation supersedes that design: Gold is now implemented purely as SQL Views over Silver, and no surrogate keys are generated anywhere in Gold. This section, and every section referencing Gold objects throughout this document, describes the current View-based implementation.

Why Views Instead of Physical Gold Tables

- Gold provides a business-ready semantic/analytical layer without owning its own copy of the data.
- Views avoid unnecessary duplication of data already governed in Silver.
- Business logic and analytical shaping is centralized in one place.
- Analytical consumers such as Power BI query a simplified, presentation-ready structure.
- Any change in Silver is reflected automatically through the Views — no separate refresh/load step for Gold.
- Gold remains lightweight and flexible, with no additional storage or ETL step to maintain.

Gold Objects (Current Implementation)

- 5 dimension Views: gold.v_dim_employee, gold.v_dim_position, gold.v_dim_manager, gold.v_dim_performance, gold.v_dim_recruitment.
- 1 fact View: gold.v_fact_hr_snapshot.

Important Structural Decisions

- Employee Status is included inside gold.v_dim_employee (via a LEFT JOIN from silver.emp to silver.emp_status). There is no separate Employee Status Gold view.
- Department is included inside gold.v_dim_position (via a LEFT JOIN from silver.position to silver.department). There is no separate Department Gold view.
- gold.v_dim_recruitment has no recruitment_id column in the current implementation; its business key is RecruitmentSource.

Keys: there are no surrogate keys in Gold. The keys used in relationships are the business keys carried forward from Silver, while Primary Keys are defined in the Silver tables. Relationships inside Gold are logical relationships based on those business keys — no Primary Key or Identity column is defined inside any Gold View.

Fact View Population

gold.v_fact_hr_snapshot is populated logically through a SELECT with LEFT JOIN from silver.hr_snapshot to the dimension Gold Views. Using LEFT JOIN preserves every source row even when a dimension lookup does not (yet) match, instead of silently dropping it.

Built-In Validation

- Row-count reconciliation between silver.hr_snapshot and gold.v_fact_hr_snapshot.

- Queries confirming no fact rows exist without a matching Employee or Position.
- A consistency check between EmpStatusID / employment status and date_termination across Silver and Bronze.
- A structural sanity check confirming Gold exposes the required Silver fields and that the object names consumed by Power BI match the current View names.

Expected Output

Reusable Gold Views that preserve star-schema logic and add no physical storage and no new surrogate keys.

Note: a Date dimension exists in the analytical/conceptual design to support Hire Date, Termination Date, and Last Performance Review Date. Any actual SQL implementation of it should be treated as a separate step unless its View/table already exists in the current scripts — it does not currently exist as a Gold SQL object (see the Date Dimension discussion under the Power BI Semantic Model section).

Historical note: the previous design built physical Gold tables with surrogate keys. The current implementation has been updated to match the revised design: Gold is now Views only over Silver, with no surrogate keys in Gold. The current code in Appendix A is the authoritative implementation reference.

ETL Pipeline

Stage	Input	Processing	Output	Business Purpose
Bronze	Raw CSV	BULK INSERT, no transformation	bronze.hr_data	Preserve an auditable original copy
Silver	bronze.hr_data	Type casting, standardization, deduplication, conformance	8 normalized tables	One trusted source for HR reporting
Gold	8 Silver tables	Analytical View definitions over Silver	5 dimension Views + 1 fact View	Answer HR business questions efficiently
Power BI	Gold layer	Import, Power Query renaming, DAX measures	4-page report	Deliver self-service HR analytics

CSV [HRDataset_v14.csv] → bronze.hr_data → Silver tables → Gold Views (v_dim_employee, v_dim_position, v_dim_manager, v_dim_performance, v_dim_recruitment, v_fact_hr_snapshot) → Power BI / Ad-hoc SQL

ETL Automation (SQL Server Agent)

Purpose: remove error-prone manual re-execution of the five scripts by scheduling them as a single ordered SQL Server Agent job.

Job Steps (in execution order)

Step	Name	What It Runs
1	00_Initialize	Creates the database/schemas — guarded by IF DB_ID(...) IS NULL / IF NOT EXISTS, safe to re-run

Step	Name	What It Runs
2	01a_Bronze_Load	Drops/recreates bronze.hr_data, then loads the CSV via BULK INSERT
3	01b_Bronze_Profiling	Runs all 15 profiling checks
4	02_Silver	Rebuilds all 8 Silver tables
5	03_Gold	Rebuilds the star schema and runs validation queries

- Execution order is enforced by on_success_action = 3 ("go to the next step") on every step.
- Failure handling: on_fail_action equals 2 ("quit the job reporting failure") for every step — the pipeline never continues silently after a step fails.
- Schedule: the job ("Daily ETL") runs once a day at 09:00, starting 2026-07-25, with no end date.
- Benefits of automation: consistent, unattended rebuilds; a single point of failure detection; no manual script execution needed after initial setup.
- Monitoring: SQL Server Agent's built-in job history/notification mechanisms can be used to alert on failure (not separately configured in the delivered job).

Warning: because the source is a static, one-time CSV extract loaded from a fixed local file path, the daily schedule does not actually capture any new data — see "Future Improvements" for the recommendation to treat this path as a configurable setting.

Known gap — Agent job Step 5 is out of date: the delivered 04_sql_agent_job.sql job step "03_Gold" still embeds the historical physical-table Gold DDL (CREATE TABLE gold.dim_emp / dim_position / dim_manager / dim_department / dim_recruitment / dim_performance / dim_emp_status / fact_hr_snapshot, with IDENTITY surrogate keys), not the current gold.sql Views script. This step must be updated to run the current View-based 03_gold.sql before the scheduled job is relied on again; until then, the automated daily run does not actually build the current Gold Views layer described in this document.

Data Lineage

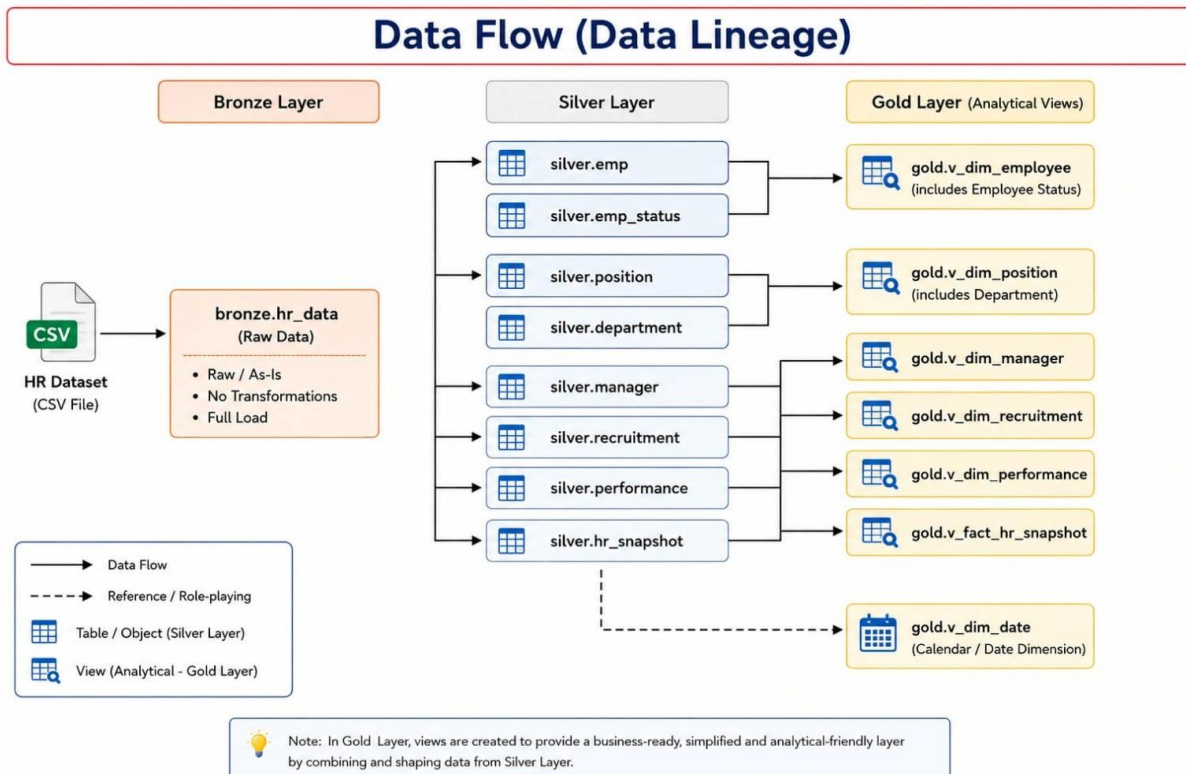


Figure 2 — Data lineage: CSV → Bronze → Silver → Gold Analytical Views, noting that the Calendar/Date dimension appears at the analytical/consumption design level.

Why Each Transformation Exists

- CSV → Bronze: preserves an unmodified audit trail of the original extract.
- Bronze → Silver: applies exactly the cleaning rules the profiling step (01b) showed to be necessary.
- Silver → Gold: reshapes the cleaned tables into an analytical presentation layer via Gold Views, retaining the business keys from Silver rather than generating surrogate keys in Gold. Date/Calendar role-playing relationships are used at the consumption layer to support Hire Date, Termination Date, and Last Performance Date.

Silver-to-Gold Mapping

Gold View	Built From (Silver)
gold.v_dim_employee	silver.emp LEFT JOIN silver.emp_status
gold.v_dim_position	silver.position LEFT JOIN silver.department
gold.v_dim_manager	silver.manager
gold.v_dim_performance	silver.performance
gold.v_dim_recruitment	silver.recruitment
gold.v_fact_hr_snapshot	silver.hr_snapshot LEFT JOIN each of the five Gold dimension Views above

Star Schema

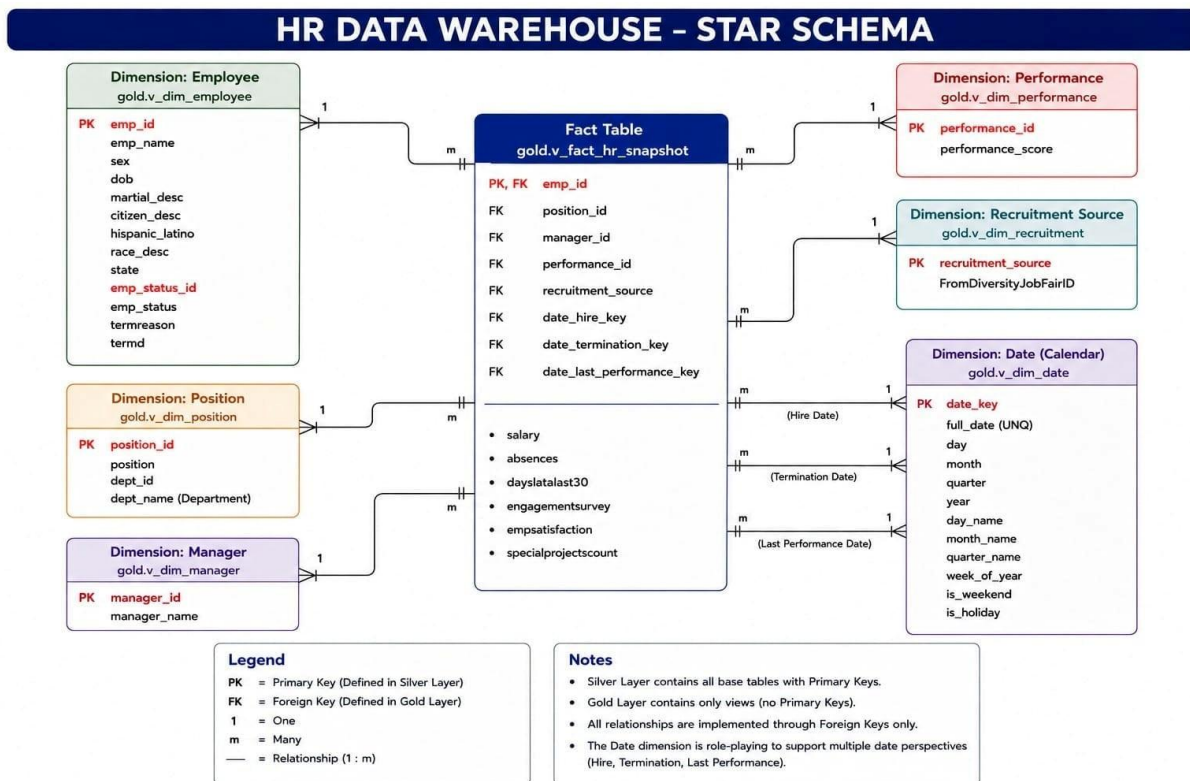


Figure 3 — Logical star schema of the current Gold layer (Gold Views over Silver).

- Fact table/view: gold.v_fact_hr_snapshot — grain: one row per employee HR-snapshot record, carrying HR measures and the business keys needed to join.
- Dimension views: gold.v_dim_employee (includes Employee Status), gold.v_dim_position (includes Department), gold.v_dim_manager, gold.v_dim_performance, and gold.v_dim_recruitment. There is no separate View for Employee Status or for Department in the current Gold layer.
- Relationships / cardinality: each logical dimension relates to the fact view in a one-to-many relationship based on business keys; Employee Status is embedded inside the Employee view, and Department is embedded inside the Position view.
- Keys: the actual primary keys are defined in the Silver tables. Gold Views contain no physical Primary Keys or surrogate keys; the join fields in Gold are the business keys carried from Silver.
- Why a star schema: it minimizes join complexity, compresses well in Power BI's VertiPaq engine, and maps directly to how business users think about the data.

Note: the diagram shows the Date Dimension/Calendar as a role-playing dimension supporting Hire Date, Termination Date, and Last Performance Date. The current implementation consumes a Calendar table inside Power BI, while the current Gold script contains no CREATE VIEW named gold.v_dim_date; this element should therefore be read as part of the analytical model rather than as a currently implemented SQL object, unless a script for it is added.

Data Dictionary — Current Gold Views

The tables below describe the columns actually exposed by each current Gold View, as defined in 03_gold.sql (Appendix A). None of these objects has a Primary Key, a Foreign Key constraint, or a surrogate/IDENTITY key — Gold is a pure presentation layer over Silver, where the real Primary Keys live.

gold.v_dim_employee

Built from silver.emp LEFT JOIN silver.emp_status ON e.empstatus_id = es.empstatus_id. Includes Employee Status.

Column	Source	Description
emp_id	silver.emp	Business key — employee ID
emp_name	silver.emp	Standardized full employee name
sex	silver.emp	Gender
dob	silver.emp	Date of birth
marital_desc	silver.emp	Marital status
citizen_desc	silver.emp	Citizenship status
hispanic_latino	silver.emp	Hispanic/Latino origin indicator
race_desc	silver.emp	Race/ethnicity description
state	silver.emp	U.S. state
empstatus_id	silver.emp_status	Business key — employment status ID
emp_status	silver.emp_status	Employment status classification (e.g., Active, Terminated)
termreason	silver.emp_status	Termination reason, if any
termd	silver.emp_status	Termination flag

gold.v_dim_position

Built from silver.position LEFT JOIN silver.department ON p.dept_id = d.dept_id. Includes Department.

Column	Source	Description
position_id	silver.position	Business key — position ID
position	silver.position	Job title
dept_id	silver.department	Business key — department ID
dept_name	silver.department	Department name

gold.v_dim_manager

Built from silver.manager.

Column	Source	Description
manager_id	silver.manager	Business key — manager ID
manager_name	silver.manager	Manager's full name

gold.v_dim_performance

Built from silver.performance.

Column	Source	Description
performance_id	silver.performance	Business key — performance rating ID
performance_score	silver.performance	Performance rating classification

gold.v_dim_recruitment

Built from silver.recruitment. There is no recruitment_id column in the current implementation.

Column	Source	Description
RecruitmentSource	silver.recruitment	Business key — recruitment channel; blank/NULL values are standardized to UNKNOWN
FromDiversityJobFairID	silver.recruitment	Flag/ID referencing a diversity job-fair source

gold.v_fact_hr_snapshot

Built from silver.hr_snapshot, LEFT JOIN to each of the five Gold dimension Views above on their respective business keys. Grain: one row per employee HR-snapshot record.

Column	Source	Description
emp_id	silver.hr_snapshot / v_dim_employee	Employee business key
emp_name	v_dim_employee	Employee name
position_id	v_dim_position	Position business key
position	v_dim_position	Job title
dept_id	v_dim_position	Department business key
dept_name	v_dim_position	Department name
manager_id	v_dim_manager	Manager business key
manager_name	v_dim_manager	Manager name
performance_id	v_dim_performance	Performance business key
performance_score	v_dim_performance	Performance rating
RecruitmentSource	v_dim_recruitment	Recruitment channel
FromDiversityJobFairID	v_dim_recruitment	Diversity job-fair flag/ID
salary	silver.hr_snapshot	Annual salary

Column	Source	Description
absences	silver.hr_snapshot	Total absences
dayslatelast30	silver.hr_snapshot	Days late in the last 30 days
engagementsurvey	silver.hr_snapshot	Engagement survey score (0–5)
empsatisfaction	silver.hr_snapshot	Satisfaction score (1–5)
specialprojectscount	silver.hr_snapshot	Count of special projects
date_hiring	silver.hr_snapshot	Hire date
date_termination	silver.hr_snapshot	Termination date
LastPerformanceReview_Date	silver.hr_snapshot	Date of the last performance review

SQL Design Decisions

Decision	Rationale
Three schemas (bronze/silver/gold)	Clear separation between raw, cleaned, and presentation layers — enforces Medallion discipline at the object-namespaces level
snake_case naming convention	Consistent, readable object and column naming throughout Silver and Gold
Primary Keys defined in Silver, not Gold	Silver is where governed, deduplicated business entities live; Gold Views inherit uniqueness from Silver rather than redefining it
No surrogate keys in Gold	Gold Views join purely on Silver's business keys — this avoids an additional identity-management layer for a presentation-only tier
LEFT JOIN when building the fact view	(not INNER JOIN) — preserves every Silver row even if a dimension lookup fails, surfacing the gap as a NULL business key rather than silently dropping data
TRY_CAST / TRY_CONVERT	Used throughout the Silver layer — prevents one malformed value from failing the entire batch load
GROUP BY + MAX(...)	For reference tables — a simple, explicit deduplication strategy for reference data sharing the same business key
No indexes beyond PK/UNIQUE constraints	The dataset size (311 rows) does not justify additional indexing at this scale

Power BI Semantic Model

- Connection: Power BI Desktop connects to the gold schema in Import mode, per the project requirement to connect to the Gold layer only.
- Data model: built around FACT_Employees (the Power-Query-renamed view of gold.v_fact_hr_snapshot) and a DIM_Date calendar table, alongside the standard Gold-layer dimensions.

Relationships

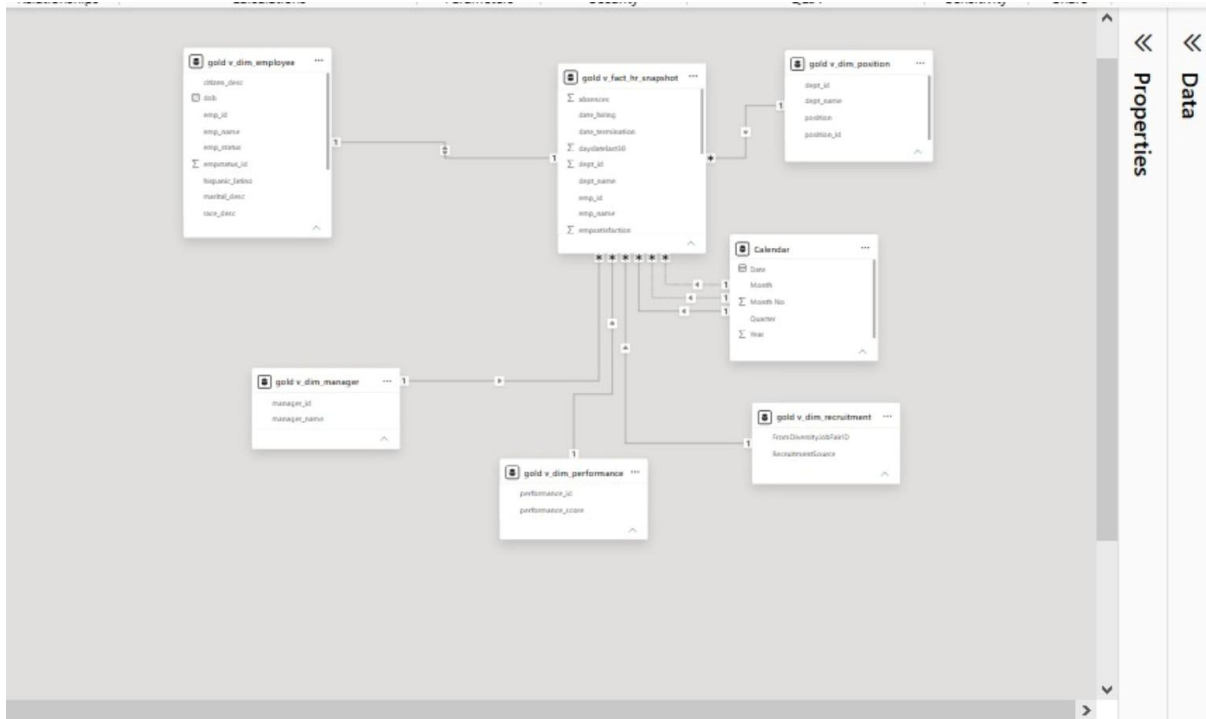


Figure 4 — Current Power BI model and relationships between the fact view, dimensions, and the Calendar table.

- Objects loaded: gold.v_fact_hr_snapshot sits at the center of the model, related to gold.v_dim_employee, gold.v_dim_position, gold.v_dim_manager, gold.v_dim_performance, and gold.v_dim_recruitment, plus a Calendar/Date table defined inside Power BI.
- Cardinality: dimensions sit on the "1" side and the fact view on the "*" side (1:* one-to-many) — the standard star-schema pattern. The Date/Calendar table uses multiple role-playing relationships where Hire / Termination / Last Performance Review dates need to be analyzed.
- Cross-filter direction: single-directional filtering from the dimensions to the fact view, with Power BI's date relationships (USERELATIONSHIP) activated in DAX measures as needed.
- Active vs. inactive relationships: the Hire Date relationship between DIM_Date and FACT_Employees is the active relationship; the Exit Date and Last Performance Review Date relationships are inactive by default and are activated per-measure via USERELATIONSHIP, which is the standard Power BI pattern for role-playing date dimensions.

Implementation note: Employee Status and Department are not independent objects in the current Gold model; the former is embedded inside gold.v_dim_employee and the latter inside gold.v_dim_position. There are also no surrogate keys in the Gold Views — relationships rely on the business keys coming from Silver.

DAX Measures Captured

The following measures were captured directly from the DAX query files saved in the delivered Power BI file (reproduced verbatim in Appendix B):

- No. of Emp = COUNT(FACT_Employees[Employee ID])
- Beginning-of-period / end-of-period headcount snapshots, using CALCULATE with hire/exit date filters and USERELATIONSHIP against DIM_Date.
- Number of employees exiting in the period / number of employees hired in the period.

- Turnover Rate = $\text{DIVIDE}(\text{exits in period}, \text{average of beginning/ending headcount})$
- Retention Rate = $1 - [\text{Turnover Rate}]$
- Time-boxed 2021 versions of the same measures, suggesting validation against a specific historical year.
- Bonus Amount = $\text{SUMX}(\text{FACT_Employees}, [\text{Annual Salary}] * [\text{Bonus \%}])$

Note on card labeling: the KPI cards on the Turnover & Retention page show cumulative, dataset-wide Turnover/Retention values (32.80% / 67.20% in the current screenshots) rather than the period-scoped DAX Turnover Rate / Retention Rate measures listed above; see "Known Deviations and Limitations" for the recommended labeling fix.

KPIs, Design, and Business Narrative

- KPIs: total headcount, gender distribution, average/total salary, min/max salary by department, overall turnover/retention rate, hires by department/gender/year, ranked recruitment sources, most/least common position, largest/smallest manager team, average satisfaction by manager.
- Dashboard design: 4 pages, each with a consistent navigation/header bar, KPI cards at the top, supporting charts below, and interactive filters.
- Filters / slicers: standard dropdown slicers plus a custom Chiclet Slicer for Department, present on every page.
- Navigation: in-report page tabs, plus a pageNavigator control and an actionButton control on several pages.
- Business narrative: each page is organized around explicit questions embedded directly in the visuals, tying the visuals directly back to the business questions in the requirements document.

Dashboard Tour

Page 1 — Workforce

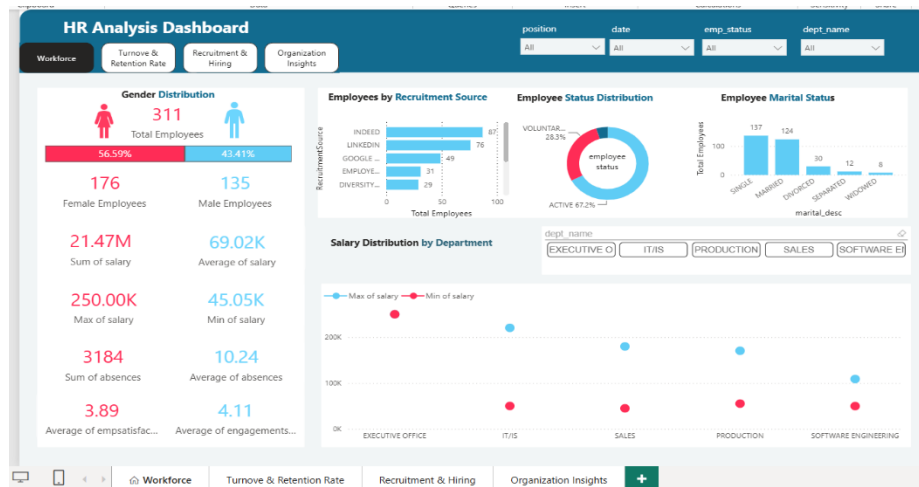


Figure 5 — Workforce page (current screenshot).

- Purpose / business question: "What does our current workforce look like?"
- KPIs: 311 total employees (176 female / 56.59%, 135 male / 43.41%); average salary \$69.02K; total salary \$21.47M; highest salary \$250.00K; lowest salary \$45.05K; total absences 3,184; average absences 10.24; average satisfaction 3.89; average engagement score 4.11.

- Visuals: employees by recruitment source (bar — Indeed 87, LinkedIn 76, Google... 49, Employee referral 31, Diversity job fair 29); Employee Status Distribution (donut — Active 67.2%, Voluntarily Terminated 28.3%, remainder Terminated for cause); Employee Marital Status (columns — Single 137, Married 124, Divorced 30, Separated 12, Widowed 8); Salary Distribution by Department (min/max combo); a Department slicer.
- Filters on this page: position, date, emp_status, dept_name.
- Why these elements: gives an executive a complete workforce census on one page before drilling into turnover or recruitment detail on later pages.
- Business insight: the workforce skews female (56.6%) and single/married (84% combined); salary ranges vary considerably across departments.

Page 2 — Turnover & Retention Rate

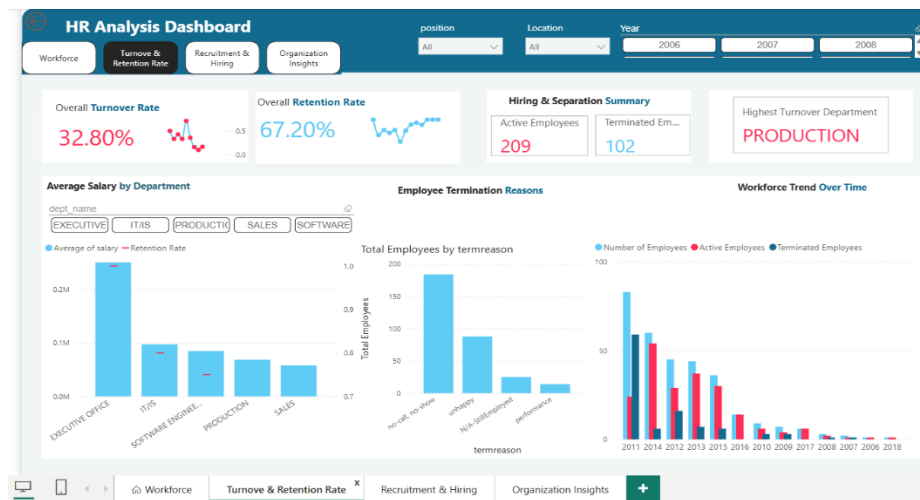


Figure 6 — Turnover & Retention Rate page (current screenshot).

- Purpose / business question: "How much attrition are we seeing, and why?"
- KPIs: overall turnover rate 32.80%; overall retention rate 67.20%; active employees 209; terminated employees 102; highest-turnover department: Production.
- Visuals: average salary by department (combo, with a retention-rate line); Employee Termination Reasons (columns — No-call/no-show ≈185 is the largest single reason, followed by Unhappy, N/A-Still Employed, and Performance); Workforce Trend Over Time (clustered columns — No. of Employees / Active Employees / Terminated Employees, 2011–2018); a Department slicer.
- Filters on this page: position, Location, Year.
- Business insight: turnover is concentrated in Production; "no-call/no-show" is the single largest driver of termination, ahead of dissatisfaction and performance-related exits.

Page 3 — Recruitment & Hiring

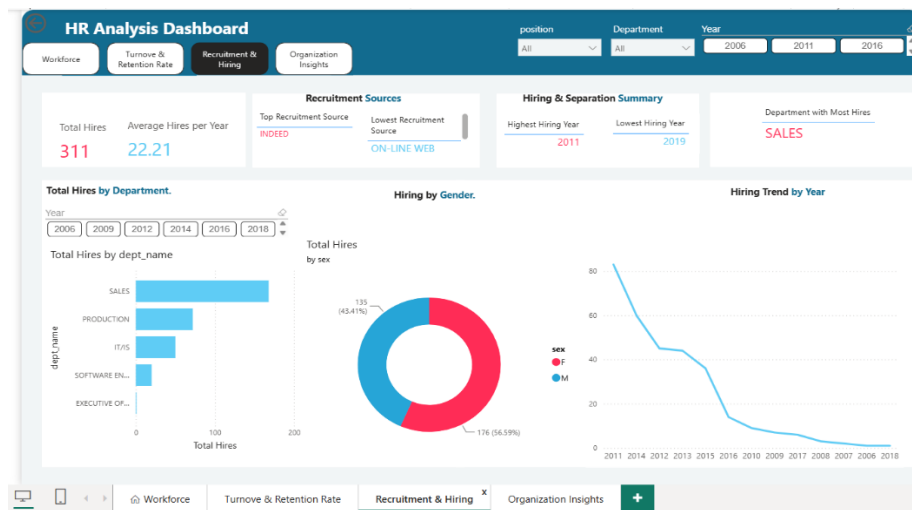


Figure 7 — Recruitment & Hiring page (current screenshot).

- Purpose / business question: "Which recruitment channels are working, and when do we hire?"
- KPIs: total hires 311; average hires per year 22.21; top recruitment source: Indeed; lowest recruitment source: On-line Web Application; highest hiring year: 2011; lowest hiring year: 2019; department with the most hires: Sales.
- Visuals: total hires by department (bar — Sales leads, ahead of Production, IT/IS, and Software Engineering, with Executive Office lowest); hiring by gender (donut — 56.59% female / 43.41% male); hiring trend by year (line, peaking in 2011 and declining through the range of years covered); a Department slicer; a Year timeline control.
- Filters on this page: position, Department, Year.
- Business insight: Indeed outperforms LinkedIn and Google as the leading recruitment source; hiring volume peaked in 2011 and has trended down across the years covered by this dataset; Sales — not Production — leads in total hires even though Production leads in headcount and turnover.

Page 4 — Organization Insights

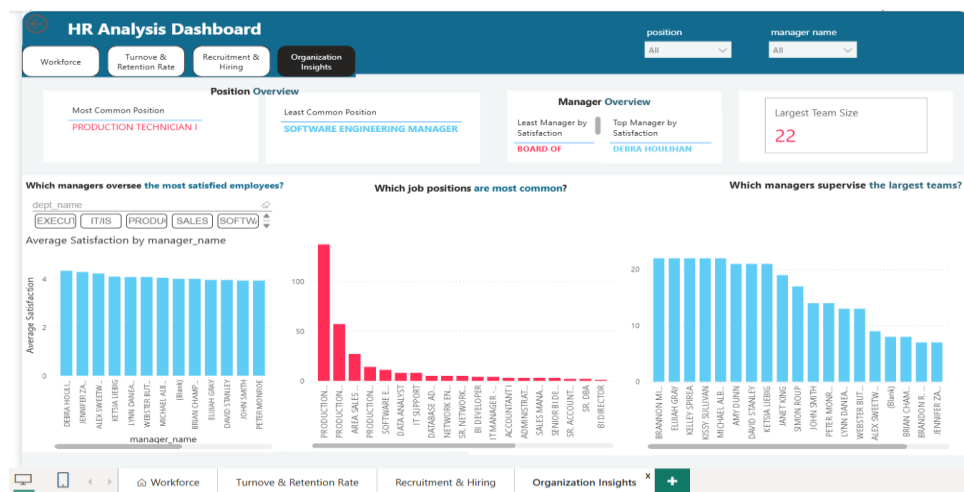


Figure 8 — Organization Insights page (current screenshot).

- Purpose / business question: "How is the organization structured, and how are managers performing?"

- KPIs: most common position: Production Technician I; least common position: Software Engineering Manager; top manager by satisfaction: Debra Houlihan; least manager by satisfaction: Board of Directors; largest team size: 22.
- Visuals: average satisfaction by manager (bar); most common job positions (bar, dominated by production roles); managers supervising the largest teams (bar); a Department slicer.
- Filters on this page: position, manager name.
- Business insight: satisfaction scores are fairly tight across managers (mostly clustering between 3.5 and 4.0), while team size varies far more widely (from 1 to 22).

Observed issue: the "Average Satisfaction by manager_name" visual on this page displays a Power BI error banner ("Something's wrong with one or more fields") in the current screenshot, though the chart still partially renders below the banner. The specific cause could not be confirmed from the image alone; see "Known Deviations and Limitations."

Business Insights

The following insights are drawn exclusively from the KPI values and visuals visible in the attached dashboard screenshots.

Recommendation	Evidence	Insight
Prioritize a targeted retention review for the Production department first	Dashboards 1, 2, 3	Production is the largest department, has the highest turnover, and leads Workforce headcount, even though Sales leads in total hires
Investigate whether this reflects an attendance policy, an onboarding gap, or an engagement gap	Dashboard 2	"No-call/no-show" is the single largest reason for termination
Redirect recruiting spend toward Indeed, pending a cost-per-hire view by source	Dashboard 3	Indeed is the strongest recruitment source; the on-line web application is the weakest
Frame this against company headcount targets given elevated cumulative turnover	Dashboards 1, 2	Hiring volume trended down every year from 2011 to 2019 in this dataset
No material gender gap visible at the hiring stage in this data	Dashboards 1, 3	The workforce is 56.6% female and 43.4% male, consistently across pages
Team size alone does not explain satisfaction differences; deeper segment analysis is needed	Dashboard 4	Manager satisfaction clusters tightly (3.5–4.0) while team size varies widely (1–22)

Known Deviations from Original Design

Documenting these deviations transparently is itself good engineering practice — a professional report should show what was actually built, not only what was originally planned.

Area	Planned Design	Actual Implementation	Why It Matters
Gold storage model	03_gold_views.sql — Views only, no physical tables	Physical TABLE objects for all 7 dimensions and the fact table	Gold data would go stale unless every ETL run rebuilds it fully; Views instead reflect Silver directly
Date dimension	gold.dim_date	No dim_date in SQL; DIM_Date is built only inside Power BI	SQL consumers other than this specific .pbix file do not have a Date dimension available
Layer terminology	The architecture diagram names each layer a "stored procedure"	Each layer is an ordinary sequential T-SQL script/batch, not a CREATE PROCEDURE object	Terminology gap between diagram and implementation; no functional impact
File portability	The delivered script is a fixed local Windows file path	The warehouse must be rebuildable by anyone who clones it	Reduces portability; must be edited before running elsewhere
Standalone script vs. job step consistency	The Agent job's "03_Gold" step still contains the historical physical-table Gold DDL (with surrogate keys), while the standalone 03_gold.sql script has been migrated to Views	The Agent job should run a full rebuild from zero using the current scripts	Re-running the scheduled job today would rebuild the outdated physical Gold tables, not the current Gold Views — see "ETL Automation"
Daily cadence vs. a static source	The source is a one-time, fixed CSV extract from a fixed local path	Automation implies continuously refreshable data	Daily scheduling against a static source
KPI card labeling	Card values (32.80% / 67.20%) reflect the cumulative dataset, not a time-boxed DAX measure	Dashboard cards labeled "Overall Turnover/Retention Rate"	KPI-card values and the period-scoped DAX measures need to be reconciled and clearly labeled
Power BI visual error	"Average Satisfaction by manager_name" shows a Power BI error banner in the current Organization Insights screenshot	The visual is expected to render without error	Cause not confirmed from the screenshot alone; should be investigated in the .pbix file directly

Data Validation

The following validation checks exist in the current SQL implementation (03_gold.sql, Appendix A):

1. Silver-to-Gold Row-Count Validation

SELECT COUNT(*) is run against both silver.hr_snapshot and gold.v_fact_hr_snapshot. This confirms the fact View is not silently dropping or duplicating rows from its Silver source.

2. Missing Employee Relationship Validation

A LEFT JOIN from gold.v_fact_hr_snapshot to gold.v_dim_employee, filtered to rows where the employee key is NULL, surfaces any fact record that cannot be resolved to an employee — this should return zero rows in a healthy load.

3. Missing Position Relationship Validation

The same pattern is applied against `gold.v_dim_position`, to confirm every fact row resolves to a valid position/department.

4. Employee Status Consistency Validation

Groups `bronze.hr_data` by `EmpStatusID` and counts `DISTINCT EmploymentStatus` values per ID, flagging any `EmpStatusID` that maps to more than one raw status label — a sign the ID-to-label mapping is not one-to-one in the source.

5. Termination-Date Consistency Validation

Joins `silver.hr_snapshot` to `silver.emp` and `silver.emp_status` and flags any employee whose status is not `ACTIVE` but whose `date_termination` is `NULL` — a business rule that a terminated employee should always carry a termination date.

Each validation exists to catch a specific, plausible failure mode of the Bronze → Silver → Gold pipeline before it reaches Power BI: dropped rows, orphaned foreign keys, inconsistent status labeling, and missing termination dates.

Challenges

- Data cleaning: reconciling numeric IDs stored as text (such as `ManagerID` as "22.0") required an explicit `FLOAT` → `INT` casting fix, discovered through profiling rather than assumed up front.
- Data quality: deciding how to handle missing `RecruitmentSource` values (standardizing to "UNKNOWN" rather than dropping them) required a deliberate trade-off between completeness and precision.
- Relationships: ensuring the Gold fact view preserves every Silver row even when a dimension lookup fails required deliberate use of `LEFT JOIN` plus explicit validation queries for `NULL` foreign keys.
- SQL: balancing simplicity (no stored procedures, no extra indexing) against enterprise-pattern expectations, appropriate for a 311-row educational dataset.
- Power BI: reconciling business-friendly Power Query column names against the underlying `snake_case` Gold schema names, and building DAX that is period-aware for turnover/retention rates.
- ETL automation: structuring the SQL Server Agent job so that the database/schema creation step can safely run whether standalone or repeatedly inside the job.
- Modeling: the decision not to implement a SQL-level Date dimension despite the original design diagrams calling for one, building it directly inside Power BI instead.

Lessons Learned

- Profiling before cleaning is not a formality — the single most impactful fix in this project (casting `ManagerID` through `FLOAT`) came directly from a profiling query.
- `TRY_CAST`/`TRY_CONVERT` combined with `LEFT JOIN` is a robust pattern for a data warehouse that degrades gracefully instead of failing outright or silently dropping rows.
- The value of a star schema is not just query performance — separating dimensions from facts maps directly onto how business stakeholders phrase their questions.
- Diagrams and documentation can drift from the actual implementation; explicitly reconciling them is itself a valuable engineering discipline.
- Automating a pipeline surfaces design questions — such as whether a daily schedule actually matches a static data source — that do not surface when scripts are run manually.

Future Improvements

- Incremental loading: move the Bronze layer from a full delete/reload pattern to an incremental upsert pattern.
- Stored procedures: wrap each layer's logic in CREATE PROCEDURE objects, matching the architecture diagram's original intent.
- Update the SQL Server Agent job's "03_Gold" step to run the current Gold Views script, replacing the outdated physical-table DDL still embedded in that step.
- Native SQL Date dimension: build gold.v_dim_date (or a physical calendar table) in T-SQL so that any BI tool inherits a proper calendar dimension, not only this specific .pbix file.
- SSIS / Azure Data Factory / Microsoft Fabric: replace the ad hoc BULK INSERT with a managed ingestion pipeline.
- Power BI Service: publish the report and configure a scheduled dataset refresh.
- Row-Level Security (RLS): restrict department managers to see only their own department's data.
- CI/CD: add automated deployment/schema scripts instead of manually running .sql files in order.
- Cloud deployment: migrate the warehouse to Azure SQL Database or Microsoft Fabric Warehouse.
- Parameterize the bulk-load path: replace the fixed local file path with a configurable variable, a network location, or cloud (blob) storage.
- Containerize the SQL Server instance and pipeline (see "Docker / Containerization") for environment consistency and easier onboarding.
- Reconcile and clearly label the Turnover/Retention KPI cards against the period-scoped DAX measures, and investigate the Power BI error on the Organization Insights satisfaction-by-manager visual.

Conclusion

This project delivers a complete, practical HR data warehouse from end to end: the raw CSV file is loaded unmodified into Bronze, cleaned and conformed in Silver through explicit transformation rules driven by profiling results, and then presented as a Gold analytical Views layer over Silver — rather than physical Gold tables or new surrogate keys — before being consumed in Power BI.

Technologies Used

- SQL Server
- T-SQL
- SSMS
- SQL Server Agent
- Power BI Desktop
- DAX
- CSV / Excel
- GitHub

References

- Human Resources Data Set (HRDataset_v14) — Kaggle, the project's data source.
- Official Microsoft SQL Server / T-SQL documentation, for BULK INSERT and TRY_CAST/TRY_CONVERT syntax.
- Official Power BI / DAX documentation, for time-intelligence patterns and USERELATIONSHIP.

Contact

For any technical questions about this project, please reach out via the GitHub repository:

- github.com/rahmahamdy590-del/HR-DataWarehouse-Project

Appendix A — Complete SQL Scripts (Current Implementation)

This appendix reproduces the complete, current content of every SQL script in this project, in execution order. Bronze-layer scripts (01a, 01b) are unchanged from the original implementation. The database-initialization (00), Silver (02), and Gold (03) scripts below are the current versions — the Gold script in particular replaces the historical physical-table design with the current View-based implementation described throughout this document. The SQL Server Agent job script (04) is reproduced as delivered; note the known gap flagged in "ETL Automation" regarding its Gold step.

00_initialize_database.sql — Current Version

► 00_initialize_database.sql

```
IF DB_ID('HR_Project') IS NULL
BEGIN
    CREATE DATABASE HR_Project;
END
GO

USE HR_Project;
GO

IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'bronze')
EXEC('CREATE SCHEMA bronze');
GO

IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'silver')
EXEC('CREATE SCHEMA silver');
GO

IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'gold')
EXEC('CREATE SCHEMA gold');
GO
```

01a_bronze_load.sql

► 01a_bronze_load.sql

```
USE HR_Project;
GO

DROP TABLE IF EXISTS bronze.hr_data;
```

```

GO

CREATE TABLE bronze.hr_data (
    Employee_Name          VARCHAR(MAX),
    EmpID                  VARCHAR(MAX),
    MarriedID              VARCHAR(MAX),
    MaritalStatusID        VARCHAR(MAX),
    GenderID               VARCHAR(MAX),
    EmpStatusID            VARCHAR(MAX),
    DeptID                 VARCHAR(MAX),
    PerfScoreID            VARCHAR(MAX),
    FromDiversityJobFairID VARCHAR(MAX),
    Salary                 VARCHAR(MAX),
    Termd                  VARCHAR(MAX),
    PositionID             VARCHAR(MAX),
    Position               VARCHAR(MAX),
    State                  VARCHAR(MAX),
    Zip                    VARCHAR(MAX),
    DOB                    VARCHAR(MAX),
    Sex                    VARCHAR(MAX),
    MaritalDesc            VARCHAR(MAX),
    CitizenDesc            VARCHAR(MAX),
    HispanicLatino         VARCHAR(MAX),
    RaceDesc               VARCHAR(MAX),
    DateofHire             VARCHAR(MAX),
    DateofTermination      VARCHAR(MAX),
    TermReason             VARCHAR(MAX),
    EmploymentStatus       VARCHAR(MAX),
    Department             VARCHAR(MAX),
    ManagerName            VARCHAR(MAX),
    ManagerID              VARCHAR(MAX),
    RecruitmentSource      VARCHAR(MAX),
    PerformanceScore       VARCHAR(MAX),
    EngagementSurvey       VARCHAR(MAX),
    EmpSatisfaction        VARCHAR(MAX),
    SpecialProjectsCount   VARCHAR(MAX),
    LastPerformanceReview_Date VARCHAR(MAX),
    DaysLateLast30        VARCHAR(MAX),
    Absences               VARCHAR(MAX)
);
GO

BULK INSERT bronze.hr_data
FROM 'C:\Users\pc\Documents\GitHub\HR-DataWarehouse-Project\HR-DataWarehouse-
Project\data\HRDataset_v14.csv'
WITH (
    FORMAT = 'CSV',
    FIRSTROW = 2,
    FIELDQUOTE = '"',
    FIELDTERMINATOR = ',',
    ROWTERMINATOR = '0x0d0a',
    CODEPAGE = '65001',
    TABLOCK
);
GO

SELECT COUNT(*) AS row_count FROM bronze.hr_data;

SELECT COUNT(*) AS column_count
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'bronze' AND TABLE_NAME = 'hr_data';

```

01b_bronze_profiling.sql

► 01b_bronze_profiling.sql

```

USE HR_Project;
GO

/*=====
1. Duplicate EmpID
=====*/
SELECT
    'hr_data' AS [Table],
    'Duplicate Employee IDs' AS [Anomaly_Type],
    COUNT(*) AS [Affected_Row_Count],
    STUFF((
        SELECT DISTINCT ', ' + TRIM(EmpID)
        FROM bronze.hr_data
        GROUP BY TRIM(EmpID)
        HAVING COUNT(*) > 1
        FOR XML PATH(''),TYPE).value('.', 'NVARCHAR(MAX)'),1,2,'') AS [Duplicate_Keys]
FROM (
    SELECT TRIM(EmpID) AS EmpID
    FROM bronze.hr_data
    GROUP BY TRIM(EmpID)
    HAVING COUNT(*)>1
) D;
GO

/*=====
2. Missing Employee Name
=====*/
SELECT
    'hr_data' AS [Table],
    'Missing Employee Name' AS [Anomaly_Type],
    COUNT(*) AS [Affected_Row_Count]
FROM bronze.hr_data
WHERE Employee_Name IS NULL
OR TRIM(Employee_Name)='';
GO

/*=====
3. Missing Department
=====*/
SELECT
    'hr_data' AS [Table],
    'Missing Department' AS [Anomaly_Type],
    COUNT(*) AS [Affected_Row_Count]
FROM bronze.hr_data
WHERE Department IS NULL
OR TRIM(Department)='';
GO

/*=====
4. Missing Position
=====*/
SELECT
    'hr_data' AS [Table],
    'Missing Position' AS [Anomaly_Type],
    COUNT(*) AS [Affected_Row_Count]
FROM bronze.hr_data
WHERE Position IS NULL
OR TRIM(Position)='';
GO

/*=====

```

```

5. Missing ManagerID
=====*/
SELECT
'hr_data' AS [Table],
'Missing ManagerID' AS [Anomaly_Type],
COUNT(*) AS [Affected_Row_Count]
FROM bronze.hr_data
WHERE ManagerID IS NULL
OR TRIM(ManagerID)='';
GO

/*=====
6. Invalid Salary
=====*/
SELECT
'hr_data' AS [Table],
'Invalid Salary (<=0)' AS [Anomaly_Type],
COUNT(*) AS [Affected_Row_Count]
FROM bronze.hr_data
WHERE TRY_CAST(Salary AS FLOAT)<=0;
GO

/*=====
7. Invalid Engagement Survey
=====*/
SELECT
'hr_data' AS [Table],
'Invalid Engagement Survey' AS [Anomaly_Type],
COUNT(*) AS [Affected_Row_Count]
FROM bronze.hr_data
WHERE TRY_CAST(EngagementSurvey AS FLOAT) NOT BETWEEN 0 AND 5;
GO

/*=====
8. Invalid Employee Satisfaction
=====*/
SELECT
'hr_data' AS [Table],
'Invalid Employee Satisfaction' AS [Anomaly_Type],
COUNT(*) AS [Affected_Row_Count]
FROM bronze.hr_data
WHERE TRY_CAST(EmpSatisfaction AS INT) NOT BETWEEN 1 AND 5;
GO

/*=====
9. Invalid Absences
=====*/
SELECT
'hr_data' AS [Table],
'Negative Absences' AS [Anomaly_Type],
COUNT(*) AS [Affected_Row_Count]
FROM bronze.hr_data
WHERE TRY_CAST(Absences AS INT)<0;
GO

/*=====
10. Invalid Days Late
=====*/
SELECT
'hr_data' AS [Table],
'Negative Days Late' AS [Anomaly_Type],
COUNT(*) AS [Affected_Row_Count]
FROM bronze.hr_data
WHERE TRY_CAST(DaysLateLast30 AS INT)<0;
GO

```

```

/*=====
11. Duplicate Employee Names
=====*/
SELECT
'hr_data' AS [Table],
'Duplicate Employee Names' AS [Anomaly_Type],
COUNT(*) AS [Affected_Row_Count],
STUFF((
SELECT DISTINCT ', '+Employee_Name
FROM bronze.hr_data
GROUP BY Employee_Name
HAVING COUNT(*)>1
FOR XML PATH(''),TYPE).value(' ','NVARCHAR(MAX)'),1,2,'') AS [Duplicate_Names]
FROM(
SELECT Employee_Name
FROM bronze.hr_data
GROUP BY Employee_Name
HAVING COUNT(*)>1
)X;
GO

/*=====
12. Invalid ManagerID (Orphan Records)
-- FIX: ManagerID/EmpID may be stored as text with a decimal point
-- (e.g. "1.0"), which TRY_CAST(... AS INT) directly rejects and
-- returns NULL for. Casting ManagerID through FLOAT first, and
-- TRIM-ing EmpID before casting, keeps this join reliable.
=====*/
SELECT
'hr_data' AS [Table],
'Orphan Manager IDs' AS [Anomaly_Type],
COUNT(*) AS [Affected_Row_Count],
STUFF((
SELECT DISTINCT ', '+TRIM(e2.ManagerID)
FROM bronze.hr_data e2
LEFT JOIN bronze.hr_data m
ON TRY_CAST(TRY_CAST(e2.ManagerID AS FLOAT) AS INT) = TRY_CAST(TRIM(m.EmpID) AS INT)
WHERE e2.ManagerID IS NOT NULL
AND TRIM(e2.ManagerID)<>' '
AND m.EmpID IS NULL
FOR XML PATH(''),TYPE).value(' ','NVARCHAR(MAX)'),1,2,'') AS [Missing_ManagerIDs]
FROM bronze.hr_data e
LEFT JOIN bronze.hr_data m
ON TRY_CAST(TRY_CAST(e.ManagerID AS FLOAT) AS INT) = TRY_CAST(TRIM(m.EmpID) AS INT)
WHERE e.ManagerID IS NOT NULL
AND TRIM(e.ManagerID)<>' '
AND m.EmpID IS NULL;
GO

/*=====
13. Invalid Hire Date
=====*/
SELECT
'hr_data' AS [Table],
'Invalid Hire Date' AS [Anomaly_Type],
COUNT(*) AS [Affected_Row_Count]
FROM bronze.hr_data
WHERE TRY_CONVERT(date,DateofHire) IS NULL;
GO

/*=====
14. Invalid Termination Date
=====*/
SELECT
'hr_data' AS [Table],

```

```

'Invalid Termination Date' AS [Anomaly_Type],
COUNT(*) AS [Affected_Row_Count]
FROM bronze.hr_data
WHERE DateofTermination IS NOT NULL
AND TRIM(DateofTermination)<>' '
AND TRY_CONVERT(date,DateofTermination) IS NULL;
GO

/*=====
15. Missing Performance Score
=====*/
SELECT
'hr_data' AS [Table],
'Missing Performance Score' AS [Anomaly_Type],
COUNT(*) AS [Affected_Row_Count]
FROM bronze.hr_data
WHERE PerformanceScore IS NULL
OR TRIM(PerformanceScore)='';
GO

```

02_silver_transformation.sql — Current Version

► 02_silver_transformation.sql

```

USE HR_Project;
GO
DROP TABLE IF EXISTS silver.hr_snapshot;
DROP TABLE IF EXISTS silver.emp;
DROP TABLE IF EXISTS silver.position;
DROP TABLE IF EXISTS silver.manager;
DROP TABLE IF EXISTS silver.department;
DROP TABLE IF EXISTS silver.recruitment;
DROP TABLE IF EXISTS silver.performance;
DROP TABLE IF EXISTS silver.emp_status;
GO

CREATE TABLE silver.emp
(
    emp_id INT PRIMARY KEY,
    empstatus_id INT NOT NULL,
    emp_name VARCHAR(100) NOT NULL,
    sex VARCHAR(20),
    dob DATE,
    marital_desc VARCHAR(50),
    citizen_desc VARCHAR(50),
    hispanic_latino VARCHAR(20),
    race_desc VARCHAR(50),
    state VARCHAR(50)
);

CREATE TABLE silver.position
(
    position_id INT PRIMARY KEY,
    position VARCHAR(100),
    dept_id INT NOT NULL
);

CREATE TABLE silver.manager
(
    manager_id INT PRIMARY KEY,
    manager_name VARCHAR(100)
);

```

```

CREATE TABLE silver.department
(
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(100) NOT NULL
);

CREATE TABLE silver.recruitment
(
    RecruitmentSource VARCHAR(100) PRIMARY KEY,
    FromDiversityJobFairID INT
);

CREATE TABLE silver.performance
(
    performance_id INT PRIMARY KEY,
    performance_score VARCHAR(50)
);

CREATE TABLE silver.emp_status
(
    empstatus_id INT PRIMARY KEY,
    emp_status VARCHAR(100),
    termreason VARCHAR(255),
    termd BIT
);

CREATE TABLE silver.hr_snapshot
(
    salary DECIMAL(18,2) CHECK(salary>=0),
    absences INT CHECK(absences>=0),
    dayslatelast30 INT CHECK(dayslatelast30>=0),
    engagementsurvey DECIMAL(5,2) CHECK(engagementsurvey BETWEEN 0 AND 5),
    empsatisfaction INT CHECK(empsatisfaction BETWEEN 1 AND 5),
    specialprojectscount INT,
    date_hiring DATE,
    date_termination DATE,
    LastPerformanceReview_Date DATE,
    RecruitmentSource VARCHAR(100),
    position_id INT,
    manager_id INT,
    performance_id INT,
    emp_id INT
);
GO

INSERT INTO silver.emp
(
    emp_id, empstatus_id, emp_name, sex, dob, marital_desc,
    citizen_desc, hispanic_latino, race_desc, state
)
SELECT DISTINCT
    TRY_CAST(TRIM(EmpID) AS INT),
    TRY_CAST(TRIM(EmpStatusID) AS INT),
    CASE
        WHEN CHARINDEX(',', Employee_Name) > 0 THEN
            UPPER(
                LTRIM(SUBSTRING(Employee_Name, CHARINDEX(',', Employee_Name) + 1,
                    LEN(Employee_Name)))
                + ',' +
                LTRIM(LEFT(Employee_Name, CHARINDEX(',', Employee_Name) - 1))
            )
        ELSE UPPER(TRIM(Employee_Name))
    END,
    UPPER(TRIM(Sex)),
    TRY_CONVERT(DATE, DOB),

```

```

        UPPER(TRIM(MaritalDesc)),
        UPPER(TRIM(CitizenDesc)),
        UPPER(TRIM(HispanicLatino)),
        UPPER(TRIM(RaceDesc)),
        UPPER(TRIM(State))
FROM bronze.hr_data;
GO

INSERT INTO silver.position (position_id, position, dept_id)
SELECT TRY_CAST(TRIM(PositionID) AS INT), MAX(UPPER(TRIM(Position))), MAX(TRY_CAST(TRIM(DeptID)
AS INT))
FROM bronze.hr_data
WHERE TRY_CAST(TRIM(PositionID) AS INT) IS NOT NULL
GROUP BY TRY_CAST(TRIM(PositionID) AS INT);
GO

INSERT INTO silver.manager (manager_id, manager_name)
SELECT TRY_CAST(TRY_CAST(TRIM(ManagerID) AS FLOAT) AS INT), MAX(UPPER(TRIM(ManagerName)))
FROM bronze.hr_data
WHERE TRY_CAST(TRY_CAST(TRIM(ManagerID) AS FLOAT) AS INT) IS NOT NULL
GROUP BY TRY_CAST(TRY_CAST(TRIM(ManagerID) AS FLOAT) AS INT);
GO

INSERT INTO silver.department (dept_id, dept_name)
SELECT TRY_CAST(TRIM(DeptID) AS INT), MAX(UPPER(TRIM(Department)))
FROM bronze.hr_data
WHERE TRY_CAST(TRIM(DeptID) AS INT) IS NOT NULL
GROUP BY TRY_CAST(TRIM(DeptID) AS INT);
GO

INSERT INTO silver.recruitment (RecruitmentSource, FromDiversityJobFairID)
SELECT
    CASE WHEN RecruitmentSource IS NULL OR TRIM(RecruitmentSource)='' THEN 'UNKNOWN'
        ELSE UPPER(TRIM(RecruitmentSource)) END,
    MAX(COALESCE(TRY_CAST(FromDiversityJobFairID AS INT),0))
FROM bronze.hr_data
GROUP BY CASE WHEN RecruitmentSource IS NULL OR TRIM(RecruitmentSource)='' THEN 'UNKNOWN'
        ELSE UPPER(TRIM(RecruitmentSource)) END;
GO

INSERT INTO silver.performance (performance_id, performance_score)
SELECT TRY_CAST(TRIM(PerfScoreID) AS INT), MAX(UPPER(TRIM(PerformanceScore)))
FROM bronze.hr_data
WHERE TRY_CAST(TRIM(PerfScoreID) AS INT) IS NOT NULL
GROUP BY TRY_CAST(TRIM(PerfScoreID) AS INT);
GO

INSERT INTO silver.emp_status (empstatus_id, emp_status, termreason, termd)
SELECT
    TRY_CAST(TRIM(EmpStatusID) AS INT),
    CASE
        WHEN TRY_CAST(TRIM(EmpStatusID) AS INT) IN (1,2,3) THEN 'ACTIVE'
        WHEN TRY_CAST(TRIM(EmpStatusID) AS INT) = 4 THEN 'TERMINATED FOR CAUSE'
        WHEN TRY_CAST(TRIM(EmpStatusID) AS INT) = 5 THEN 'VOLUNTARILY TERMINATED'
        ELSE MAX(UPPER(TRIM(EmploymentStatus)))
    END,
    MAX(TRIM(TermReason)),
    CAST(MAX(CAST(Termd AS INT)) AS BIT)
FROM bronze.hr_data
WHERE TRY_CAST(TRIM(EmpStatusID) AS INT) IS NOT NULL
GROUP BY TRY_CAST(TRIM(EmpStatusID) AS INT);
GO

INSERT INTO silver.hr_snapshot

```

```
(
    salary, absences, dayslatelast30, engagementsurvey, empsatisfaction,
    specialprojectscount, date_hiring, date_termination, LastPerformanceReview_Date,
    RecruitmentSource, position_id, manager_id, performance_id, emp_id
)
SELECT
    COALESCE(TRY_CAST(Salary AS DECIMAL(18,2)),0),
    COALESCE(TRY_CAST(Absences AS INT),0),
    COALESCE(TRY_CAST(DaysLateLast30 AS INT),0),
    COALESCE(TRY_CAST(EngagementSurvey AS DECIMAL(5,2)),0),
    COALESCE(TRY_CAST(EmpSatisfaction AS INT),0),
    COALESCE(TRY_CAST(SpecialProjectsCount AS INT),0),
    TRY_CONVERT(DATE, DateofHire),
    TRY_CONVERT(DATE, DateofTermination),
    TRY_CONVERT(DATE, LastPerformanceReview_Date),
    CASE WHEN RecruitmentSource IS NULL OR TRIM(RecruitmentSource)='' THEN 'UNKNOWN'
         ELSE UPPER(TRIM(RecruitmentSource)) END,
    TRY_CAST(TRIM(PositionID) AS INT),
    TRY_CAST(TRY_CAST(TRIM(ManagerID) AS FLOAT) AS INT),
    TRY_CAST(TRIM(PerfScoreID) AS INT),
    TRY_CAST(TRIM(EmpID) AS INT)
FROM bronze.hr_data;
GO
```

03_gold.sql — Current Gold Views Version

► 03_gold.sql

```
USE HR_Project;
GO
DROP VIEW IF EXISTS gold.v_fact_hr_snapshot;
DROP VIEW IF EXISTS gold.v_dim_employee;
DROP VIEW IF EXISTS gold.v_dim_position;
DROP VIEW IF EXISTS gold.v_dim_manager;
DROP VIEW IF EXISTS gold.v_dim_performance;
DROP VIEW IF EXISTS gold.v_dim_recruitment;
GO

CREATE VIEW gold.v_dim_employee AS
SELECT
    e.emp_id,
    e.emp_name,
    e.sex,
    e.dob,
    e.marital_desc,
    e.citizen_desc,
    e.hispanic_latino,
    e.race_desc,
    e.state,
    es.empstatus_id,
    es.emp_status,
    es.termreason,
    es.termid
FROM silver.emp e
LEFT JOIN silver.emp_status es
    ON e.empstatus_id = es.empstatus_id;
GO

CREATE VIEW gold.v_dim_position AS
SELECT
    p.position_id,
    p.position,
    d.dept_id,
    d.dept_name
```

```

FROM silver.position p
LEFT JOIN silver.department d
    ON p.dept_id = d.dept_id;
GO

CREATE VIEW gold.v_dim_manager AS
SELECT manager_id, manager_name
FROM silver.manager;
GO

CREATE VIEW gold.v_dim_performance AS
SELECT performance_id, performance_score
FROM silver.performance;
GO

CREATE VIEW gold.v_dim_recruitment AS
SELECT RecruitmentSource, FromDiversityJobFairID
FROM silver.recruitment;
GO

CREATE VIEW gold.v_fact_hr_snapshot AS
SELECT
    e.emp_id,
    e.emp_name,
    p.position_id,
    p.position,
    p.dept_id,
    p.dept_name,
    m.manager_id,
    m.manager_name,
    pr.performance_id,
    pr.performance_score,
    r.RecruitmentSource,
    r.FromDiversityJobFairID,
    s.salary,
    s.absences,
    s.dayslatelast30,
    s.engagementsurvey,
    s.empsatisfaction,
    s.specialprojectscount,
    s.date_hiring,
    s.date_termination,
    s.LastPerformanceReview_Date
FROM silver.hr_snapshot s
LEFT JOIN gold.v_dim_employee e ON s.emp_id = e.emp_id
LEFT JOIN gold.v_dim_position p ON s.position_id = p.position_id
LEFT JOIN gold.v_dim_manager m ON s.manager_id = m.manager_id
LEFT JOIN gold.v_dim_performance pr ON s.performance_id = pr.performance_id
LEFT JOIN gold.v_dim_recruitment r ON s.RecruitmentSource = r.RecruitmentSource;
GO

-- Validation 1: row-count preservation
SELECT COUNT(*) AS Silver_Count FROM silver.hr_snapshot;
SELECT COUNT(*) AS Gold_Count FROM gold.v_fact_hr_snapshot;

-- Validation 2: missing employee matches
SELECT *
FROM gold.v_fact_hr_snapshot f
LEFT JOIN gold.v_dim_employee e ON f.emp_id = e.emp_id
WHERE e.emp_id IS NULL;

-- Validation 3: missing position matches
SELECT *
FROM gold.v_fact_hr_snapshot f

```

```

LEFT JOIN gold.v_dim_position p ON f.position_id = p.position_id
WHERE p.position_id IS NULL;

-- Validation 4: EmpStatusID consistency
SELECT EmpStatusID, COUNT(DISTINCT EmploymentStatus) AS Different_Statuses
FROM bronze.hr_data
GROUP BY EmpStatusID
HAVING COUNT(DISTINCT EmploymentStatus) > 1;

-- Validation 5: terminated employee must have a termination date
SELECT e.emp_name, es.emp_status, h.date_termination
FROM silver.hr_snapshot h
JOIN silver.emp e ON h.emp_id = e.emp_id
JOIN silver.emp_status es ON e.empstatus_id = es.empstatus_id
WHERE es.emp_status <> 'ACTIVE'
      AND h.date_termination IS NULL;

```

04_sql_agent_job.sql

Reproduced as delivered. As flagged in "ETL Automation," job Step 5 ("03_Gold") below still contains the historical physical-table Gold DDL rather than the current Views script, and should be updated before this job is relied on for a production rebuild.

► 04_sql_agent_job.sql

```

USE [msdb]
GO

/***** Object: Job [HR ETL Pipeline]    Script Date: 7/25/2026 8:58:08 AM *****/
BEGIN TRANSACTION
DECLARE @ReturnCode INT
SELECT @ReturnCode = 0
/***** Object: JobCategory [[Uncategorized (Local)]]    Script Date: 7/25/2026 8:58:08 AM *****/
IF NOT EXISTS (SELECT name FROM msdb.dbo.syscategories WHERE name=N'[Uncategorized (Local)]' AND category_class=1)
BEGIN
EXEC @ReturnCode = msdb.dbo.sp_add_category @class=N'JOB', @type=N'LOCAL',
@name=N'[Uncategorized (Local)]'
IF (@@ERROR <> 0 OR @ReturnCode <> 0) GOTO QuitWithRollback

END

DECLARE @jobId BINARY(16)
EXEC @ReturnCode =  msdb.dbo.sp_add_job @job_name=N'HR ETL Pipeline',
        @enabled=1,
        @notify_level_eventlog=0,
        @notify_level_email=0,
        @notify_level_netsend=0,
        @notify_level_page=0,
        @delete_level=0,
        @description=N'No description available.',
        @category_name=N'[Uncategorized (Local)]',
        @owner_login_name=N'sa', @job_id = @jobId OUTPUT
IF (@@ERROR <> 0 OR @ReturnCode <> 0) GOTO QuitWithRollback
/***** Object: Step [00_Initialize]    Script Date: 7/25/2026 8:58:09 AM *****/
EXEC @ReturnCode = msdb.dbo.sp_add_jobstep @job_id=@jobId, @step_name=N'00_Initialize',
        @step_id=1,
        @cmdexec_success_code=0,
        @on_success_action=3,
        @on_success_step_id=0,
        @on_fail_action=2,
        @on_fail_step_id=0,

```

```

        @retry_attempts=0,
        @retry_interval=0,
        @os_run_priority=0, @subsystem='NTSQL',
        @command=N'IF DB_ID(''HR_Project'') IS NULL
BEGIN
    CREATE DATABASE HR_Project;
END
GO

USE HR_Project;
GO

IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = ''bronze'')
    EXEC(''CREATE SCHEMA bronze'');
GO

IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = ''silver'')
    EXEC(''CREATE SCHEMA silver'');
GO

IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = ''gold'')
    EXEC(''CREATE SCHEMA gold'');
GO',
        @database_name=N'master',
        @flags=0
IF (@@ERROR <> 0 OR @ReturnCode <> 0) GOTO QuitWithRollback
/***** Object: Step [01a_Bronze_Load]    Script Date: 7/25/2026 8:58:09 AM *****/
EXEC @ReturnCode = msdb.dbo.sp_add_jobstep @job_id=@jobId, @step_name=N'01a_Bronze_Load',
        @step_id=2,
        @cmdexec_success_code=0,
        @on_success_action=3,
        @on_success_step_id=0,
        @on_fail_action=2,
        @on_fail_step_id=0,
        @retry_attempts=0,
        @retry_interval=0,
        @os_run_priority=0, @subsystem='NTSQL',
        @command=N'USE HR_Project;
GO

DROP TABLE IF EXISTS bronze.hr_data;
GO

CREATE TABLE bronze.hr_data (
    Employee_Name          VARCHAR(MAX),
    EmpID                  VARCHAR(MAX),
    MarriedID              VARCHAR(MAX),
    MaritalStatusID        VARCHAR(MAX),
    GenderID               VARCHAR(MAX),
    EmpStatusID            VARCHAR(MAX),
    DeptID                 VARCHAR(MAX),
    PerfScoreID            VARCHAR(MAX),
    FromDiversityJobFairID VARCHAR(MAX),
    Salary                 VARCHAR(MAX),
    TermID                 VARCHAR(MAX),
    PositionID             VARCHAR(MAX),
    Position               VARCHAR(MAX),
    State                  VARCHAR(MAX),
    Zip                    VARCHAR(MAX),
    DOB                    VARCHAR(MAX),
    Sex                    VARCHAR(MAX),
    MaritalDesc            VARCHAR(MAX),
    CitizenDesc            VARCHAR(MAX),
    HispanicLatino         VARCHAR(MAX),
    RaceDesc               VARCHAR(MAX),

```

```

        DateofHire                VARCHAR(MAX),
        DateofTermination          VARCHAR(MAX),
        TermReason                 VARCHAR(MAX),
        EmploymentStatus           VARCHAR(MAX),
        Department                 VARCHAR(MAX),
        ManagerName                VARCHAR(MAX),
        ManagerID                 VARCHAR(MAX),
        RecruitmentSource           VARCHAR(MAX),
        PerformanceScore           VARCHAR(MAX),
        EngagementSurvey            VARCHAR(MAX),
        EmpSatisfaction             VARCHAR(MAX),
        SpecialProjectsCount        VARCHAR(MAX),
        LastPerformanceReview_Date VARCHAR(MAX),
        DaysLateLast30             VARCHAR(MAX),
        Absences                   VARCHAR(MAX)
    );
GO

BULK INSERT bronze.hr_data
FROM 'C:\Users\pc\Documents\GitHub\HR-DataWarehouse-Project\HR-DataWarehouse-
Project\data\HRDataset_v14.csv'
WITH (
    FORMAT = 'CSV',
    FIRSTROW = 2,
    FIELDQUOTE = '"""',
    FIELDTERMINATOR = ',',
    ROWTERMINATOR = '0x0d0a',
    CODEPAGE = '65001',
    TABLOCK
);
GO

SELECT COUNT(*) AS row_count FROM bronze.hr_data;

SELECT COUNT(*) AS column_count
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'bronze' AND TABLE_NAME = 'hr_data';',
        @database_name=N'HR_Project',
        @flags=0
IF (@@ERROR <> 0 OR @ReturnCode <> 0) GOTO QuitWithRollback
/***** Object: Step [01b_Bronze_Profiling] Script Date: 7/25/2026 8:58:09 AM *****/
EXEC @ReturnCode = msdb.dbo.sp_add_jobstep @job_id=@jobId, @step_name=N'01b_Bronze_Profiling',
        @step_id=3,
        @cmdexec_success_code=0,
        @on_success_action=3,
        @on_success_step_id=0,
        @on_fail_action=2,
        @on_fail_step_id=0,
        @retry_attempts=0,
        @retry_interval=0,
        @os_run_priority=0, @subsystem=N'TSQL',
        @command=N'USE HR_Project;

GO
SET ANSI_NULLS ON;
SET QUOTED_IDENTIFIER ON;
GO
/*=====
1. Duplicate EmpID
=====*/
SELECT
    'hr_data' AS [Table],
    'Duplicate Employee IDs' AS [Anomaly_Type],
    COUNT(*) AS [Affected_Row_Count],
    STUFF((
        SELECT DISTINCT ' ', ' ' + TRIM(EmpID)

```

```

        FROM bronze.hr_data
        GROUP BY TRIM(EmpID)
        HAVING COUNT(*) > 1
        FOR XML PATH(''),TYPE).value(''.', 'NVARCHAR(MAX)'),1,2, '') AS [Duplicate_Keys]
FROM (
    SELECT TRIM(EmpID) AS EmpID
    FROM bronze.hr_data
    GROUP BY TRIM(EmpID)
    HAVING COUNT(*)>1
) D;
GO

/*=====
2. Missing Employee Name
=====*/
SELECT
    'hr_data' AS [Table],
    'Missing Employee Name' AS [Anomaly_Type],
    COUNT(*) AS [Affected_Row_Count]
FROM bronze.hr_data
WHERE Employee_Name IS NULL
OR TRIM(Employee_Name)='';
GO

/*=====
3. Missing Department
=====*/
SELECT
    'hr_data' AS [Table],
    'Missing Department' AS [Anomaly_Type],
    COUNT(*) AS [Affected_Row_Count]
FROM bronze.hr_data
WHERE Department IS NULL
OR TRIM(Department)='';
GO

/*=====
4. Missing Position
=====*/
SELECT
    'hr_data' AS [Table],
    'Missing Position' AS [Anomaly_Type],
    COUNT(*) AS [Affected_Row_Count]
FROM bronze.hr_data
WHERE Position IS NULL
OR TRIM(Position)='';
GO

/*=====
5. Missing ManagerID
=====*/
SELECT
    'hr_data' AS [Table],
    'Missing ManagerID' AS [Anomaly_Type],
    COUNT(*) AS [Affected_Row_Count]
FROM bronze.hr_data
WHERE ManagerID IS NULL
OR TRIM(ManagerID)='';
GO

/*=====
6. Invalid Salary
=====*/
SELECT
    'hr_data' AS [Table],

```

```

'Invalid Salary (<=0)' AS [Anomaly_Type],
COUNT(*) AS [Affected_Row_Count]
FROM bronze.hr_data
WHERE TRY_CAST(Salary AS FLOAT)<=0;
GO

/*=====
7. Invalid Engagement Survey
=====*/
SELECT
'hr_data' AS [Table],
'Invalid Engagement Survey' AS [Anomaly_Type],
COUNT(*) AS [Affected_Row_Count]
FROM bronze.hr_data
WHERE TRY_CAST(EngagementSurvey AS FLOAT) NOT BETWEEN 0 AND 5;
GO

/*=====
8. Invalid Employee Satisfaction
=====*/
SELECT
'hr_data' AS [Table],
'Invalid Employee Satisfaction' AS [Anomaly_Type],
COUNT(*) AS [Affected_Row_Count]
FROM bronze.hr_data
WHERE TRY_CAST(EmpSatisfaction AS INT) NOT BETWEEN 1 AND 5;
GO

/*=====
9. Invalid Absences
=====*/
SELECT
'hr_data' AS [Table],
'Negative Absences' AS [Anomaly_Type],
COUNT(*) AS [Affected_Row_Count]
FROM bronze.hr_data
WHERE TRY_CAST(Absences AS INT)<0;
GO

/*=====
10. Invalid Days Late
=====*/
SELECT
'hr_data' AS [Table],
'Negative Days Late' AS [Anomaly_Type],
COUNT(*) AS [Affected_Row_Count]
FROM bronze.hr_data
WHERE TRY_CAST(DaysLateLast30 AS INT)<0;
GO

/*=====
11. Duplicate Employee Names
=====*/
SELECT
'hr_data' AS [Table],
'Duplicate Employee Names' AS [Anomaly_Type],
COUNT(*) AS [Affected_Row_Count],
STUFF(
SELECT DISTINCT ' ', '+Employee_Name
FROM bronze.hr_data
GROUP BY Employee_Name
HAVING COUNT(*)>1
FOR XML PATH(''),TYPE).value('','NVARCHAR(MAX)'),1,2,'') AS [Duplicate_Names]
FROM(
SELECT Employee_Name

```

```

FROM bronze.hr_data
GROUP BY Employee_Name
HAVING COUNT(*)>1
)X;
GO

/*=====
12. Invalid ManagerID (Orphan Records)
-- FIX: ManagerID/EmpID may be stored as text with a decimal point
-- (e.g. "1.0"), which TRY_CAST(... AS INT) directly rejects and
-- returns NULL for. Casting ManagerID through FLOAT first, and
-- TRIM-ing EmpID before casting, keeps this join reliable.
=====*/
SELECT
  'hr_data' AS [Table],
  'Orphan Manager IDs' AS [Anomaly_Type],
  COUNT(*) AS [Affected_Row_Count],
  STUFF(
    SELECT DISTINCT ' ', '+TRIM(e2.ManagerID)
    FROM bronze.hr_data e2
    LEFT JOIN bronze.hr_data m
    ON TRY_CAST(TRY_CAST(e2.ManagerID AS FLOAT) AS INT) = TRY_CAST(TRIM(m.EmpID) AS INT)
    WHERE e2.ManagerID IS NOT NULL
    AND TRIM(e2.ManagerID)<>' '
    AND m.EmpID IS NULL
    FOR XML PATH(''),TYPE).value('','NVARCHAR(MAX)'),1,2,'') AS [Missing_ManagerIDs]
FROM bronze.hr_data e
LEFT JOIN bronze.hr_data m
ON TRY_CAST(TRY_CAST(e.ManagerID AS FLOAT) AS INT) = TRY_CAST(TRIM(m.EmpID) AS INT)
WHERE e.ManagerID IS NOT NULL
AND TRIM(e.ManagerID)<>' '
AND m.EmpID IS NULL;
GO

/*=====
13. Invalid Hire Date
=====*/
SELECT
  'hr_data' AS [Table],
  'Invalid Hire Date' AS [Anomaly_Type],
  COUNT(*) AS [Affected_Row_Count]
FROM bronze.hr_data
WHERE TRY_CONVERT(date,DateofHire) IS NULL;
GO

/*=====
14. Invalid Termination Date
=====*/
SELECT
  'hr_data' AS [Table],
  'Invalid Termination Date' AS [Anomaly_Type],
  COUNT(*) AS [Affected_Row_Count]
FROM bronze.hr_data
WHERE DateofTermination IS NOT NULL
AND TRIM(DateofTermination)<>' '
AND TRY_CONVERT(date,DateofTermination) IS NULL;
GO

/*=====
15. Missing Performance Score
=====*/
SELECT
  'hr_data' AS [Table],
  'Missing Performance Score' AS [Anomaly_Type],
  COUNT(*) AS [Affected_Row_Count]
FROM bronze.hr_data

```

```

WHERE PerformanceScore IS NULL
OR TRIM(PerformanceScore)='';
GO',
        @database_name=N'HR_Project',
        @flags=0
IF (@@ERROR <> 0 OR @ReturnCode <> 0) GOTO QuitWithRollback
/***** Object: Step [02_Silver] Script Date: 7/25/2026 8:58:10 AM *****/
EXEC @ReturnCode = msdb.dbo.sp_add_jobstep @job_id=@jobId, @step_name=N'02_Silver',
        @step_id=4,
        @cmexec_success_code=0,
        @on_success_action=3,
        @on_success_step_id=0,
        @on_fail_action=2,
        @on_fail_step_id=0,
        @retry_attempts=0,
        @retry_interval=0,
        @os_run_priority=0, @subsystem=N'TSQL',
        @command=N'USE HR_Project;

GO
DROP TABLE IF EXISTS silver.hr_snapshot;
DROP TABLE IF EXISTS silver.emp;
DROP TABLE IF EXISTS silver.position;
DROP TABLE IF EXISTS silver.manager;
DROP TABLE IF EXISTS silver.department;
DROP TABLE IF EXISTS silver.recruitment;
DROP TABLE IF EXISTS silver.performance;
DROP TABLE IF EXISTS silver.emp_status;
GO
-----DDL
CREATE TABLE silver.emp
(
    emp_id INT,
    emp_name VARCHAR(100),
    sex VARCHAR(20),
    dob DATE,
    marital_desc VARCHAR(50),
    citizen_desc VARCHAR(50),
    hispanic_latino VARCHAR(20),
    race_desc VARCHAR(50),
    state VARCHAR(50)
);
CREATE TABLE silver.position
(
    position_id INT,
    position VARCHAR(100)
);
CREATE TABLE silver.manager
(
    manager_id INT,
    manager_name VARCHAR(100)
);
CREATE TABLE silver.department
(
    dept_id INT,
    dept_name VARCHAR(100)
);
CREATE TABLE silver.recruitment
(
    RecruitmentSource VARCHAR(100),
    FromDiversityJobFairID INT
);
CREATE TABLE silver.performance
(
    performance_id INT,
    performance_score VARCHAR(50)
);
CREATE TABLE silver.emp_status

```

```

(
    empstatus_id INT,
    emp_status VARCHAR(100),
    termreason VARCHAR(255),
    termd BIT
);
CREATE TABLE silver.hr_snapshot
(
    salary DECIMAL(18,2),
    absences INT,
    dayslatelast30 INT,
    engagementsurvey DECIMAL(5,2),
    empsatisfaction INT,
    empstatus_id INT,
    specialprojectscount INT,
    date_hiring DATE,
    date_termination DATE,
    LastPerformanceReview_Date DATE,
    RecruitmentSource VARCHAR(100),
    position_id INT,
    manager_id INT,
    dept_id INT,
    performance_id INT,
    emp_id INT
);
GO
-----DML
INSERT INTO silver.emp
(
    emp_id,
    emp_name,
    sex,
    dob,
    marital_desc,
    citizen_desc,
    hispanic_latino,
    race_desc,
    state
)
SELECT DISTINCT
    TRY_CAST(TRIM(EmpID) AS INT),
    CASE
        WHEN CHARINDEX(',', Employee_Name) > 0 THEN
            UPPER(
                LTRIM(SUBSTRING(Employee_Name, CHARINDEX(',', Employee_Name) + 1,
                    LEN(Employee_Name)))
                + ' ' +
                LTRIM(LEFT(Employee_Name, CHARINDEX(',', Employee_Name) - 1))
            )
        ELSE
            UPPER(TRIM(Employee_Name))
    END,
    UPPER(TRIM(Sex)),
    TRY_CONVERT(DATE, DOB),
    UPPER(TRIM(MaritalDesc)),
    UPPER(TRIM(CitizenDesc)),
    UPPER(TRIM(HispanicLatino)),
    UPPER(TRIM(RaceDesc)),
    UPPER(TRIM(State))
FROM bronze.hr_data;
GO
INSERT INTO silver.position
(
    position_id,
    position
)
SELECT

```

```

        TRY_CAST(TRIM(PositionID) AS INT),
        MAX(UPPER(TRIM(Position)))
FROM bronze.hr_data
WHERE TRY_CAST(TRIM(PositionID) AS INT) IS NOT NULL
GROUP BY TRY_CAST(TRIM(PositionID) AS INT);
GO
INSERT INTO silver.manager
(
    manager_id,
    manager_name
)
SELECT
    TRY_CAST(TRY_CAST(TRIM(ManagerID) AS FLOAT) AS INT),
    MAX(UPPER(TRIM(ManagerName)))
FROM bronze.hr_data
WHERE TRY_CAST(TRY_CAST(TRIM(ManagerID) AS FLOAT) AS INT) IS NOT NULL
GROUP BY TRY_CAST(TRY_CAST(TRIM(ManagerID) AS FLOAT) AS INT);
GO
INSERT INTO silver.department
(
    dept_id,
    dept_name
)
SELECT
    TRY_CAST(TRIM(DeptID) AS INT),
    MAX(UPPER(TRIM(Department)))
FROM bronze.hr_data
WHERE TRY_CAST(TRIM(DeptID) AS INT) IS NOT NULL
GROUP BY TRY_CAST(TRIM(DeptID) AS INT);
GO

INSERT INTO silver.recruitment
(
    RecruitmentSource,
    FromDiversityJobFairID
)
SELECT
    CASE
        WHEN RecruitmentSource IS NULL OR TRIM(RecruitmentSource)='' THEN 'UNKNOWN'
        ELSE UPPER(TRIM(RecruitmentSource))
    END,
    MAX(COALESCE(TRY_CAST(FromDiversityJobFairID AS INT),0))
FROM bronze.hr_data
GROUP BY
    CASE
        WHEN RecruitmentSource IS NULL OR TRIM(RecruitmentSource)='' THEN 'UNKNOWN'
        ELSE UPPER(TRIM(RecruitmentSource))
    END;
GO

INSERT INTO silver.performance
(
    performance_id,
    performance_score
)
SELECT
    TRY_CAST(TRIM(PerfScoreID) AS INT),
    MAX(UPPER(TRIM(PerformanceScore)))
FROM bronze.hr_data
WHERE TRY_CAST(TRIM(PerfScoreID) AS INT) IS NOT NULL
GROUP BY TRY_CAST(TRIM(PerfScoreID) AS INT);
GO

INSERT INTO silver.emp_status
(
    empstatus_id,

```

```

    emp_status,
    termreason,
    termd
)
SELECT
    TRY_CAST(TRIM(EmpStatusID) AS INT),
    MAX(UPPER(TRIM(EmploymentStatus))),
    MAX(TRIM(TermReason)),
    CAST(MAX(CAST(Termd AS INT)) AS BIT)
FROM bronze.hr_data
WHERE TRY_CAST(TRIM(EmpStatusID) AS INT) IS NOT NULL
GROUP BY TRY_CAST(TRIM(EmpStatusID) AS INT);
GO
-----DML - HR_snapshot
INSERT INTO silver.hr_snapshot
(
    salary,
    absences,
    dayslatelast30,
    engagementsurvey,
    empsatisfaction,
    specialprojectscount,
    empstatus_id,
    date_hiring,
    date_termination,
    LastPerformanceReview_Date,
    RecruitmentSource,
    position_id,
    manager_id,
    dept_id,
    performance_id,
    emp_id
)
SELECT
    COALESCE(TRY_CAST(Salary AS DECIMAL(18,2)),0),
    COALESCE(TRY_CAST(Absences AS INT),0),
    COALESCE(TRY_CAST(DaysLateLast30 AS INT),0),
    COALESCE(TRY_CAST(EngagementSurvey AS DECIMAL(5,2)),0),
    COALESCE(TRY_CAST(EmpSatisfaction AS INT),0),
    COALESCE(TRY_CAST(SpecialProjectsCount AS INT),0),
    TRY_CAST(TRIM(EmpStatusID) AS INT),
    TRY_CONVERT(DATE, DateofHire),
    TRY_CONVERT(DATE, DateofTermination),
    TRY_CONVERT(DATE, LastPerformanceReview_Date),
    CASE
        WHEN RecruitmentSource IS NULL
            OR TRIM(RecruitmentSource)='' THEN 'UNKNOWN'
        ELSE UPPER(TRIM(RecruitmentSource))
    END,
    TRY_CAST(TRIM(PositionID) AS INT),
    TRY_CAST(TRY_CAST(TRIM(ManagerID) AS FLOAT) AS INT),
    TRY_CAST(TRIM(DeptID) AS INT),
    TRY_CAST(TRIM(PerfScoreID) AS INT),
    TRY_CAST(TRIM(EmpID) AS INT)
FROM bronze.hr_data;
GO
',
    @database_name=N'HR_Project',
    @flags=0
IF (@@ERROR <> 0 OR @ReturnCode <> 0) GOTO QuitWithRollback
/***** Object: Step [03_Gold] Script Date: 7/25/2026 8:58:10 AM *****/
EXEC @ReturnCode = msdb.dbo.sp_add_jobstep @job_id=@jobId, @step_name=N'03_Gold',
    @step_id=5,
    @cmdexec_success_code=0,
    @on_success_action=1,
    @on_success_step_id=0,
    @on_fail_action=2,

```

```

        @on_fail_step_id=0,
        @retry_attempts=0,
        @retry_interval=0,
        @os_run_priority=0, @subsystem=N'TSQL',
        @command=N'USE HR_Project;

GO
DROP TABLE IF EXISTS gold.fact_hr_snapshot;
DROP TABLE IF EXISTS gold.dim_emp;
DROP TABLE IF EXISTS gold.dim_position;
DROP TABLE IF EXISTS gold.dim_manager;
DROP TABLE IF EXISTS gold.dim_department;
DROP TABLE IF EXISTS gold.dim_recruitment;
DROP TABLE IF EXISTS gold.dim_performance;
DROP TABLE IF EXISTS gold.dim_emp_status;
GO
-----DDL - DIMENSIONS (surrogate key = sk_<dimension>, NOT NULL by definition of IDENTITY
PK; business key = UNIQUE, so no duplicate surrogate rows for the same source key)
CREATE TABLE gold.dim_emp
(
    sk_emp          INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    emp_id          INT UNIQUE,
    emp_name        VARCHAR(100),
    sex             VARCHAR(20),
    dob             DATE,
    marital_desc    VARCHAR(50),
    citizen_desc    VARCHAR(50),
    hispanic_latino VARCHAR(20),
    race_desc       VARCHAR(50),
    state           VARCHAR(50)
);
CREATE TABLE gold.dim_position
(
    sk_position INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    position_id INT UNIQUE,
    position     VARCHAR(100)
);
CREATE TABLE gold.dim_manager
(
    sk_manager  INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    manager_id  INT UNIQUE,
    manager_name VARCHAR(100)
);
CREATE TABLE gold.dim_department
(
    sk_department INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    dept_id       INT UNIQUE,
    dept_name     VARCHAR(100)
);
CREATE TABLE gold.dim_recruitment
(
    sk_recruitment      INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    RecruitmentSource   VARCHAR(100) UNIQUE,
    FromDiversityJobFairID INT
);
CREATE TABLE gold.dim_performance
(
    sk_performance  INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    performance_id  INT UNIQUE,
    performance_score VARCHAR(50)
);
CREATE TABLE gold.dim_emp_status
(
    sk_emp_status INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    empstatus_id  INT UNIQUE,
    emp_status    VARCHAR(100),
    termreason    VARCHAR(255),
    termd         BIT

```

```

);
GO
-----DDL - FACT (surrogate key + FKs to dimension surrogate keys)
CREATE TABLE gold.fact_hr_snapshot
(
    sk_hr_snapshot          INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    sk_emp                  INT REFERENCES gold.dim_emp(sk_emp),
    sk_position             INT REFERENCES gold.dim_position(sk_position),
    sk_manager              INT REFERENCES gold.dim_manager(sk_manager),
    sk_department           INT REFERENCES gold.dim_department(sk_department),
    sk_performance          INT REFERENCES gold.dim_performance(sk_performance),
    sk_emp_status           INT REFERENCES gold.dim_emp_status(sk_emp_status),
    sk_recruitment          INT REFERENCES gold.dim_recruitment(sk_recruitment),
    salary                  DECIMAL(18,2),
    absences                INT,
    dayslatelast30          INT,
    engagementsurvey        DECIMAL(5,2),
    empsatisfaction         INT,
    specialprojectscount    INT,
    date_hiring             DATE,
    date_termination        DATE,
    LastPerformanceReview_Date DATE
);
GO
-----DML - DIMENSIONS (DISTINCT + surrogate keys are NOT listed/inserted -> SQL Server
generates them.)
INSERT INTO gold.dim_emp
(
    emp_id,
    emp_name,
    sex,
    dob,
    marital_desc,
    citizen_desc,
    hispanic_latino,
    race_desc,
    state
)
SELECT DISTINCT
    emp_id,
    emp_name,
    sex,
    dob,
    marital_desc,
    citizen_desc,
    hispanic_latino,
    race_desc,
    state
FROM silver.emp;
GO
INSERT INTO gold.dim_position
(
    position_id,
    position
)
SELECT DISTINCT
    position_id,
    position
FROM silver.position;
GO
INSERT INTO gold.dim_manager
(
    manager_id,
    manager_name
)
SELECT DISTINCT
    manager_id,

```

```

        manager_name
FROM silver.manager;
GO
INSERT INTO gold.dim_department
(
    dept_id,
    dept_name
)
SELECT DISTINCT
    dept_id,
    dept_name
FROM silver.department;
GO
INSERT INTO gold.dim_recruitment
(
    RecruitmentSource,
    FromDiversityJobFairID
)
SELECT DISTINCT
    RecruitmentSource,
    FromDiversityJobFairID
FROM silver.recruitment;
GO
INSERT INTO gold.dim_performance
(
    performance_id,
    performance_score
)
SELECT DISTINCT
    performance_id,
    performance_score
FROM silver.performance;
GO
INSERT INTO gold.dim_emp_status
(
    empstatus_id,
    emp_status,
    termreason,
    termd
)
SELECT DISTINCT
    empstatus_id,
    emp_status,
    termreason,
    termd
FROM silver.emp_status;
GO
-----DML - FACT_HR_SNAPSHOT (look up each dimension''s surrogate key via its business key)
INSERT INTO gold.fact_hr_snapshot
(
    sk_emp,
    sk_position,
    sk_manager,
    sk_department,
    sk_performance,
    sk_emp_status,
    sk_recruitment,
    salary,
    absences,
    dayslatelast30,
    engagementsurvey,
    empsatisfaction,
    specialprojectscount,
    date_hiring,
    date_termination,
    LastPerformanceReview_Date
)

```

```

SELECT
    de.sk_emp,
    dp.sk_position,
    dm.sk_manager,
    dd.sk_department,
    dpf.sk_performance,
    des.sk_emp_status,
    dr.sk_recruitment,
    s.salary,
    s.absences,
    s.dayslatelast30,
    s.engagementsurvey,
    s.empsatisfaction,
    s.specialprojectscount,
    s.date_hiring,
    s.date_termination,
    s.LastPerformanceReview_Date
FROM silver.hr_snapshot s
LEFT JOIN gold.dim_emp      de ON de.emp_id      = s.emp_id
LEFT JOIN gold.dim_position dp ON dp.position_id  = s.position_id
LEFT JOIN gold.dim_manager dm ON dm.manager_id   = s.manager_id
LEFT JOIN gold.dim_department dd ON dd.dept_id   = s.dept_id
LEFT JOIN gold.dim_performance dpf ON dpf.performance_id = s.performance_id
LEFT JOIN gold.dim_emp_status des ON des.empstatus_id = s.empstatus_id
LEFT JOIN gold.dim_recruitment dr ON dr.RecruitmentSource = s.RecruitmentSource;
GO

-----VALIDATION
SELECT
    'Silver HR Snapshot' AS Layer,
    COUNT(*) AS Row_Count
FROM silver.hr_snapshot
UNION ALL
SELECT
    'Gold Fact HR Snapshot',
    COUNT(*)
FROM gold.fact_hr_snapshot;
GO

-- Rows in the fact table missing any surrogate key
SELECT * FROM gold.fact_hr_snapshot
WHERE sk_emp IS NULL OR sk_position IS NULL OR sk_department IS NULL
      OR sk_manager IS NULL OR sk_performance IS NULL OR sk_emp_status IS NULL
      OR sk_recruitment IS NULL;
GO

-- Drill down: raw bronze rows behind any sk_manager gaps found above
-- (dynamic - no hardcoded IDs, re-runs correctly on any future load)
SELECT b.EmpID, b.Employee_Name, b.ManagerID, b.ManagerName, b.Position
FROM bronze.hr_data b
WHERE TRY_CAST(TRIM(b.EmpID) AS INT) IN (
    SELECT de.emp_id
    FROM gold.fact_hr_snapshot f
    JOIN gold.dim_emp de ON de.sk_emp = f.sk_emp
    WHERE f.sk_manager IS NULL
);
GO

-----check for null
SELECT
    emp_id,
    manager_id
FROM silver.hr_snapshot
WHERE emp_id IN
(
    10277,
    10184,
    10154,

```

```

10136,
10214,
10077,
10011,
10071
);
SELECT *
FROM silver.manager
WHERE manager_name = 'WEBSTER BUTLER';

SELECT
    EmpID,
    ManagerID,
    ManagerName
FROM bronze.hr_data
WHERE ManagerName = 'Webster Butler';',
        @database_name=N'HR_Project',
        @flags=0
IF (@@ERROR <> 0 OR @ReturnCode <> 0) GOTO QuitWithRollback
EXEC @ReturnCode = msdb.dbo.sp_update_job @job_id = @jobId, @start_step_id = 1
IF (@@ERROR <> 0 OR @ReturnCode <> 0) GOTO QuitWithRollback
EXEC @ReturnCode = msdb.dbo.sp_add_jobschedule @job_id=@jobId, @name=N'Daily ETL',
        @enabled=1,
        @freq_type=4,
        @freq_interval=1,
        @freq_subday_type=1,
        @freq_subday_interval=0,
        @freq_relative_interval=0,
        @freq_recurrence_factor=0,
        @active_start_date=20260725,
        @active_end_date=99991231,
        @active_start_time=90000,
        @active_end_time=235959,
        @schedule_uid=N'94ca6bf7-e69c-47ba-89a7-c4f75a32c450'
IF (@@ERROR <> 0 OR @ReturnCode <> 0) GOTO QuitWithRollback
EXEC @ReturnCode = msdb.dbo.sp_add_jobserver @job_id = @jobId, @server_name = N'(local)'
IF (@@ERROR <> 0 OR @ReturnCode <> 0) GOTO QuitWithRollback
COMMIT TRANSACTION
GOTO EndSave
QuitWithRollback:
    IF (@@TRANCOUNT > 0) ROLLBACK TRANSACTION
EndSave:
GO

```

Appendix B — Full DAX Query Definitions

The following are the complete, unmodified DAX query/measure definitions, captured directly from the DAX query files saved in the delivered Power BI file.

DAX Query File: Query 1.dax

► Query 1.dax

```

// Learn more about DAX queries at https://aka.ms/dax-queries

// Here is a sample DAX query from your model, click 'Run'
// Try other DAX queries by right clicking a table, column, or measure in the data pane and
// choosing one from 'Quick queries'
EVALUATE
    TOPN(100, 'FACT_Employees')

```

DAX Query File: Query 2.dax

► Query 2.dax

```

DEFINE
    MEASURE 'DIM_Date'[Bonus Amount] = SUMX(
        FACT_Employees,
        FACT_Employees[Annual Salary] * FACT_Employees[Bonus %]
    )
    MEASURE 'Measure'[No. of Emp] = COUNT(FACT_Employees[Employee ID])
    MEASURE 'Measure'[No. of Emp Beginning of 2021] = CALCULATE(
        [No. of Emp],
        FACT_Employees[Hire Date] <= DATE(2021, 1, 1),
        FACT_Employees[Exit Date] > DATE(2021, 1, 1) ||
        ISBLANK(FACT_Employees[Exit Date])
    )
    MEASURE 'Measure'[No. of Emp End of 2021] = CALCULATE(
        [No. of Emp],
        FACT_Employees[Hire Date] <= DATE(2021, 12, 31),
        FACT_Employees[Exit Date] > DATE(2021, 12, 31) ||
        ISBLANK(FACT_Employees[Exit Date])
    )
    MEASURE 'Measure'[No. of Emp Exit in 2021] = CALCULATE(
        [No. of Emp],
        FACT_Employees[Exit Date] >= DATE(2021, 1, 1),
        FACT_Employees[Exit Date] <= DATE(2021, 12, 31)
    )
    MEASURE 'Measure'[Turnover Rate 2021] = VAR AvgEmp = DIVIDE(
        [No. of Emp Beginning of 2021] + [No. of Emp End of 2021],
        2
    )
    VAR ExitEmp = [No. of Emp Exit in 2021]
    RETURN
        DIVIDE(
            ExitEmp,
            AvgEmp
        )

    // Divide([No. of Emp Exit in 2021], DIVIDE([No. of Emp Beginning of 2021]+[No. of Emp
    End of 2021], 2))
    MEASURE 'Measure'[No. of Emp Beginning of Period] = CALCULATE(
        [No. of Emp],
        FACT_Employees[Hire Date] <= MIN(DIM_Date[Date]),
        FACT_Employees[Exit Date] > MIN(DIM_Date[Date]) ||
        ISBLANK(FACT_Employees[Exit Date]),
        USERRELATIONSHIP(
            DIM_Date[Date],
            FACT_Employees[Exit Date]
        ),
        ALL(DIM_Date)
    )
    MEASURE 'Measure'[No. of Emp End of Period] = CALCULATE(
        [No. of Emp],
        FACT_Employees[Hire Date] <= MAX(DIM_Date[Date]),
        FACT_Employees[Exit Date] > MAX(DIM_Date[Date]) ||
        ISBLANK(FACT_Employees[Exit Date]),
        USERRELATIONSHIP(
            DIM_Date[Date],
            FACT_Employees[Exit Date]
        ),
        ALL(DIM_Date)
    )

```

```

    )
    MEASURE 'Measure'[No. of Emp Exit in Period] = CALCULATE(
        [No. of Emp] + 0,
        FACT_Employees[Exit Date] >= MIN(DIM_Date[Date]),
        FACT_Employees[Exit Date] <= MAX(DIM_Date[Date]),
        USERELATIONSHIP(
            DIM_Date[Date],
            FACT_Employees[Exit Date]
        ),
        ALL(DIM_Date)
    )
    MEASURE 'Measure'[Turnover Rate] = VAR AvgEmp = DIVIDE(
        [No. of Emp Beginning of Period] + [No. of Emp End of Period],
        2
    )
    VAR ExitEmp = [No. of Emp Exit in Period]
    RETURN
        DIVIDE(
            ExitEmp,
            AvgEmp
        )
    MEASURE 'Measure'[Retention Rate] = 1 - [Turnover Rate]
    MEASURE 'Measure'[No. of Emp Beginning of LY] = CALCULATE(
        [No. of Emp],
        FACT_Employees[Hire Date] <= MIN(DIM_Date[Date]),
        DIM_Date[Year] = MAX(DIM_Date[Year]) - 1,
        FACT_Employees[Exit Date] > MIN(DIM_Date[Date]) ||
        ISBLANK(FACT_Employees[Exit Date]),
        USERELATIONSHIP(
            DIM_Date[Date],
            FACT_Employees[Exit Date]
        ),
        ALL(DIM_Date)
    )
    MEASURE 'Measure'[No. of Emp Hired in Period] = CALCULATE(
        [No. of Emp] + 0,
        FACT_Employees[Hire Date] >= MIN(DIM_Date[Date]),
        FACT_Employees[Hire Date] <= MAX(DIM_Date[Date]),
        ALL(DIM_Date)
    )
)

EVALUATE
    SUMMARIZECOLUMNS(
        "Bonus Amount", 'DIM_Date'[Bonus Amount],
        "No. of Emp", 'Measure'[No. of Emp],
        "No. of Emp Beginning of 2021", 'Measure'[No. of Emp Beginning of 2021],
        "No. of Emp End of 2021", 'Measure'[No. of Emp End of 2021],
        "No. of Emp Exit in 2021", 'Measure'[No. of Emp Exit in 2021],
        "Turnover Rate 2021", 'Measure'[Turnover Rate 2021],
        "No. of Emp Beginning of Period", 'Measure'[No. of Emp Beginning of Period],
        "No. of Emp End of Period", 'Measure'[No. of Emp End of Period],
        "No. of Emp Exit in Period", 'Measure'[No. of Emp Exit in Period],
        "Turnover Rate", 'Measure'[Turnover Rate],
        "Retention Rate", 'Measure'[Retention Rate],
        "No. of Emp Beginning of LY", 'Measure'[No. of Emp Beginning of LY],
        "No. of Emp Hired in Period", 'Measure'[No. of Emp Hired in Period]
    )
)

```

DAX Query File: Query 3.dax

► Query 3.dax

```

DEFINE
    MEASURE 'Measure'[No. of Emp] = COUNT(FACT_Employees[Employee ID])
    MEASURE 'Measure'[No. of Emp Beginning of 2021] = CALCULATE(

```

```

        [No. of Emp],
        FACT_Employees[Hire Date] <= DATE(2021, 1, 1),
        FACT_Employees[Exit Date] > DATE(2021, 1, 1) ||
ISBLANK(FACT_Employees[Exit Date])
    )
    MEASURE 'Measure'[No. of Emp End of 2021] = CALCULATE(
        [No. of Emp],
        FACT_Employees[Hire Date] <= DATE(2021, 12, 31),
        FACT_Employees[Exit Date] > DATE(2021, 12, 31) ||
ISBLANK(FACT_Employees[Exit Date])
    )
    MEASURE 'Measure'[No. of Emp Exit in 2021] = CALCULATE(
        [No. of Emp],
        FACT_Employees[Exit Date] >= DATE(2021, 1, 1),
        FACT_Employees[Exit Date] <= DATE(2021, 12, 31)
    )
    MEASURE 'Measure'[Turnover Rate 2021] = VAR AvgEmp = DIVIDE(
        [No. of Emp Beginning of 2021] + [No. of Emp End of 2021],
        2
    )
    VAR ExitEmp = [No. of Emp Exit in 2021]
    RETURN
        DIVIDE(
            ExitEmp,
            AvgEmp
        )
    MEASURE 'Measure'[No. of Emp Beginning of Period] = CALCULATE(
        [No. of Emp],
        FACT_Employees[Hire Date] <= MIN(DIM_Date[Date]),
        FACT_Employees[Exit Date] > MIN(DIM_Date[Date]) ||
ISBLANK(FACT_Employees[Exit Date]),
        USERRELATIONSHIP(
            DIM_Date[Date],
            FACT_Employees[Exit Date]
        ),
        ALL(DIM_Date)
    )
    MEASURE 'Measure'[No. of Emp End of Period] = CALCULATE(
        [No. of Emp],
        FACT_Employees[Hire Date] <= MAX(DIM_Date[Date]),
        FACT_Employees[Exit Date] > MAX(DIM_Date[Date]) ||
ISBLANK(FACT_Employees[Exit Date]),
        USERRELATIONSHIP(
            DIM_Date[Date],
            FACT_Employees[Exit Date]
        ),
        ALL(DIM_Date)
    )
    MEASURE 'Measure'[No. of Emp Exit in Period] = CALCULATE(
        [No. of Emp] + 0,
        FACT_Employees[Exit Date] >= MIN(DIM_Date[Date]),
        FACT_Employees[Exit Date] <= MAX(DIM_Date[Date]),
        USERRELATIONSHIP(
            DIM_Date[Date],
            FACT_Employees[Exit Date]
        ),
        ALL(DIM_Date)
    )
    MEASURE 'Measure'[Turnover Rate] = VAR AvgEmp = DIVIDE(
        [No. of Emp Beginning of Period] + [No. of Emp End of Period],
        2
    )
    VAR ExitEmp = [No. of Emp Exit in Period]
    RETURN
        DIVIDE(
            ExitEmp,
            AvgEmp

```

```

    )
    MEASURE 'Measure'[Retention Rate] = 1 - [Turnover Rate]
    MEASURE 'Measure'[No. of Emp Begining of LY] = CALCULATE(
        [No. of Emp],
        FACT_Employees[Hire Date] <= MIN(DIM_Date[Date]),
        DIM_Date[Year] = MAX(DIM_Date[Year]) - 1,
        FACT_Employees[Exit Date] > MIN(DIM_Date[Date]) ||
ISBLANK(FACT_Employees[Exit Date]),
        USERELATIONSHIP(
            DIM_Date[Date],
            FACT_Employees[Exit Date]
        ),
        ALL(DIM_Date)
    )
    MEASURE 'Measure'[No. of Emp Hired in Period] = CALCULATE(
        [No. of Emp] + 0,
        FACT_Employees[Hire Date] >= MIN(DIM_Date[Date]),
        FACT_Employees[Hire Date] <= MAX(DIM_Date[Date]),
        ALL(DIM_Date)
    )
)

EVALUATE
    SUMMARIZECOLUMNS(
        "No. of Emp", [No. of Emp],
        "No. of Emp Begining of 2021", [No. of Emp Begining of 2021],
        "No. of Emp End of 2021", [No. of Emp End of 2021],
        "No. of Emp Exit in 2021", [No. of Emp Exit in 2021],
        "Turnover Rate 2021", [Turnover Rate 2021],
        "No. of Emp Begining of Period", [No. of Emp Begining of Period],
        "No. of Emp End of Period", [No. of Emp End of Period],
        "No. of Emp Exit in Period", [No. of Emp Exit in Period],
        "Turnover Rate", [Turnover Rate],
        "Retention Rate", [Retention Rate],
        "No. of Emp Begining of LY", [No. of Emp Begining of LY],
        "No. of Emp Hired in Period", [No. of Emp Hired in Period]
    )

```